



LiDAR SLAM with Multiple Object Tracking(MOT) for Autonomous Driving

ME5400 Project

April 14, 2026

Contents

1	Introduction and Background	1
2	Related Work Analysis	4
2.1	From Dynamic-Point Rejection to Dynamic-Aware Estimation	5
2.2	Recent SLAMMOT Systems and the Move Toward Joint Optimization	6
2.3	Pure 3D MOT Background and the Choice of Tracking Backbone	7
2.4	Comparison and Remaining Gap	7
3	Overall System Architecture, PointPillars / MCTrack, and Data Flow	8
3.1	FAST-LIO Front-End	9
3.2	PointPillars Detection Module	11
3.3	MCTrack Module and Data-Flow Details	12
3.4	Temporal Alignment, Coordinate Frames, and Failure Modes	14
4	Engineering Implementation and System Integration	14
4.1	Overall Code Organization	14
4.2	Online System Design	15
4.3	Result Archiving and the Reproducible Experimental Pipeline	18
4.4	ROS Topics, Message Interfaces, and Integration Difficulties	18
5	Dynamic Weight Optimization	19
5.1	Design Motivation and Code Location	19
5.2	Weight Computation	20
5.3	From Tracked Objects to Point-Level Weights	21
5.4	Boundary Cases and Parameter Sensitivity	22
6	Joint Backend Optimization	22
6.1	The Ego-Only Sliding-Window Problem	22
6.2	Residual Modeling and the Gating Strategy	23
6.3	Window Update Logic and Practical Behavior	24
6.4	Value and Limitations of the Joint Backend	25
7	Experimental Analysis	26

7.1	Experimental Setup and Result Sources	26
7.2	Online Experiments: Aggregate Results	27
7.3	Integrated Analysis and Failure Modes	29
7.4	From Observation to Explanation	30
7.5	Experimental Fairness and Reporting Discipline	30
8	Conclusion and Future Plan	31
	Appendix: All Online Experimental Figures	35

Abstract

This report presents a code-grounded account of the ME5400 course project on dynamic-scene LiDAR SLAM with multiple-object tracking. The system combines FAST-LIO for LiDAR-inertial ego-motion estimation, PointPillars for 3D object detection, and MCTrack for online trajectory association and motion-state estimation. The central idea is to form a bidirectional loop instead of a one-way cascade: FAST-LIO poses stabilize MCTrack association, while tracked-object states are converted into soft dynamic weights in FAST-LIO and into object-consistency factors in an ego-only sliding-window backend.

All quantitative discussion is based on archived online results under Results/ for seven KITTI Tracking sequences: 0003, 0006, 0007, 0009, 0010, 0013, and 0020. The report emphasizes both algorithmic intent and engineering realization, including ROS interfaces, time synchronization, pose buffering, standardized result archiving, and failure-mode analysis. Across the seven sequences, the front-end configuration FAST-LIO+MCTrack consistently outperforms the pure FAST-LIO baseline in ATE RMSE, showing that the most reliable gain comes from the front-end closed loop of pose-assisted tracking and tracking-informed dynamic-point down-weighting. The joint backend can help on some drift-dominated sequences, but it is less stable and can fail when velocity extrapolation, timestamp alignment, or object gating are unreliable; Sequence 0009 is the clearest example. The overall conclusion is intentionally modest: this project is not a state-of-the-art joint SLAMMOT system, but a reproducible engineering prototype that validates the feasibility of a localization-tracking feedback loop and clarifies the transition toward future object-centric joint optimization.

Keywords: FAST-LIO; PointPillars; MCTrack; dynamic-scene SLAM; multi-object tracking; object-centric perception; ROS reproducibility

1 Introduction and Background

Localization and perception in dynamic road scenes impose a contradiction that is easy to state but difficult to resolve in a full system. LiDAR SLAM and LiDAR-inertial odometry are most reliable when the majority of observed geometry is static, repeatedly visible, and reusable over time. Their front-end scan matching, local-map maintenance, and backend consistency all depend on the idea that a point observed now corresponds to a surface that is still geometrically meaningful in later frames. By contrast, autonomous driving perception cares most about the exact objects that violate this assumption: surrounding vehicles, pedestrians, cyclists, and other traffic participants. In other words, the regions that are semantically most important for decision making are often the regions that are geometrically most dangerous for classical SLAM. This conflict has motivated a long line of work on dynamic-scene SLAM, from early visual systems

that detect and filter moving regions to more recent SLAMMOT pipelines that jointly reason about ego motion and object trajectories [4, 5, 6, 10, 13].

The negative effect of dynamic objects on LiDAR SLAM is not limited to the simple statement that “moving points are outliers.” There are at least three coupled degradation channels. First, **map contamination** occurs when points from moving vehicles are inserted into a local or global map as if they were static surfaces. Once that happens, the system can accumulate ghost structures, duplicate facades, or smeared object trails that later scans mistakenly register against. Second, **registration degradation** arises even before those points are permanently fused into the map. If a large fraction of point-to-plane residuals is generated by moving objects, the local optimizer is forced to fit inconsistent geometry, which can bias the estimated pose in the current frame. Third, **drift accumulation** amplifies both problems over time: a slightly biased frame-to-frame correction distorts the local map, which then worsens later corrections, eventually producing a persistent trajectory offset. Dynamic scenes therefore do not merely add random noise; they alter the structure of the optimization problem itself.

Existing dynamic-scene SLAM systems typically respond in one of three ways. A first group tries to reject dynamic points entirely through semantic segmentation, motion consistency tests, or robust loss functions. This strategy is often effective when dynamic objects occupy a modest portion of the field of view, but it can discard too much useful geometry in traffic scenes where large vehicles dominate the scan. A second group tries to separate static and dynamic layers more explicitly, such as weighted background fusion in dense RGB-D pipelines or semantic foreground filtering in visual SLAM [4, 5, 6]. A third and newer group treats object states as first-class variables and seeks some degree of joint optimization between ego motion and tracked targets [9, 10, 11, 12, 13]. The practical question for this project is where a semester-scale engineering system should enter this design space.

Multi-object tracking (MOT) faces an opposite but equally important dependency. A tracker is often described as operating on detections, but 3D detections alone are not enough to provide stable motion estimates across frames. When the ego platform itself moves, every detection must be interpreted through ego-motion compensation; otherwise, an object that is stationary in the world can appear to move, and an object that moves smoothly can appear to jump. This affects not only association but also derived quantities such as fused velocity, acceleration, and track confidence. If the pose stream is noisy, a tracker’s world-frame motion estimate becomes noisy as well. In turn, if the tracker cannot maintain coherent IDs and velocity estimates, it cannot provide reliable feedback to the localization stack. The relationship is therefore fundamentally bidirectional: better localization improves tracking, and better tracking can improve localization by identifying which points should be trusted less or by supplying additional object-level constraints.

This project is built around that bidirectional intuition. The system uses FAST-LIO [1] as the LiDAR-inertial localization backbone, PointPillars [2] as the 3D detector, and MCTrack as

the online tracker that produces track IDs, fused velocities, accelerations, and trajectory lengths. FAST-LIO publishes both optimized odometry and IMU-predicted motion. A bridge node converts these outputs into the pose format required by MCTrack. PointPillars publishes time-stamped 3D detections in a project-defined ROS message. MCTrack consumes detections and poses, performs motion-compensated association, and publishes tracked objects. Those tracked objects are then used in two ways. First, they are transformed into soft dynamic weights that reduce the influence of points inside dynamic bounding boxes in the next FAST-LIO scan. Second, they are fed into an ego-only joint backend that tests whether stable object observations can provide additional sliding-window constraints on the ego trajectory. This design does not yet constitute a unified factor graph, but it already forms a complete information loop.

Two design choices deserve brief justification. The first is the use of **soft down-weighting** rather than hard deletion of points inside tracked dynamic boxes, because hard removal is brittle under false positives, box jitter, occlusion, and temporary stops. The second is the choice to start from a **loosely coupled engineering loop** rather than a tightly coupled object-centric graph. A full joint graph would be theoretically cleaner, but it would also require substantially more factor design, confidence modeling, marginalization, and debugging effort than a course schedule can support.

The project accordingly pursues four linked objectives. The **research objective** is to test whether localization and tracking can form a meaningful feedback loop in dynamic driving scenes without first building a full unified optimizer. The **engineering objective** is to construct a runnable ROS pipeline whose components, message contracts, startup order, result logging, and evaluation scripts are all explicit and reproducible. The **evaluation objective** is to determine which part of the loop delivers the most stable gain: the front-end weighting loop, the backend correction stage, or both. The **diagnostic objective** is to identify the specific failure modes that prevent the current system from behaving like a robust joint SLAMMOT solution.

In more concrete terms, the contributions of the current report and implementation can be summarized as follows.

- A complete ROS-based integration of FAST-LIO, PointPillars, MCTrack, and an ego-only backend, with project-defined detection and tracked-object messages rather than ad hoc private interfaces.
- A front-end closed loop in which FAST-LIO poses stabilize MCTrack, while MCTrack outputs are converted into point-level soft weights inside FAST-LIO preprocessing.
- A reproducible experiment framework with one-click baseline versus optimized runs, standardized archiving under Results/, TUM-format trajectory recording, and unified quantitative evaluation.
- A failure-aware analysis showing that the front-end loop is the most stable source of im-

provement, whereas the current backend remains fragile under velocity extrapolation and timing uncertainty.

The choice of KITTI Tracking as the main validation dataset is deliberate. KITTI provides synchronized LiDAR, IMU, calibration, and ground-truth vehicle motion for real urban driving scenarios [3], while the Tracking split contains the dynamic vehicles, trajectory variation, and background changes needed to test a localization-tracking loop. The seven online sequences used here, namely 0003, 0006, 0007, 0009, 0010, 0013, and 0020, were selected because they cover different drift levels, dynamic intensity, and relative-trajectory spans, and because the repository already contains complete archived online runs for them under `Results/`. This report therefore does not claim a full-dataset benchmark; it analyzes the implemented system on sequences whose logs, plots, and trajectory files can be directly reproduced and inspected.

This positioning leads to an important scope statement. **This report is not written as a claim of state-of-the-art performance.** It documents an engineering prototype that validates the feasibility of a bidirectional localization-tracking loop, demonstrates a full experiment pipeline, and makes its failure cases visible rather than hiding them. That scope is appropriate for a course project because integration quality, repeatability, and diagnosis are themselves valuable outcomes. A system that runs end to end, can be replayed on multiple sequences, and exposes exactly where and why it fails provides a stronger foundation for future research than a superficially tighter formulation that was never debugged into a dependable pipeline.

2 Related Work Analysis

To position the current project more carefully, related work must be discussed along two lines. The first is **dynamic-scene SLAM**, which focuses on detecting, filtering, or jointly modeling moving regions so that localization and mapping remain stable. The second is **3D MOT for autonomous driving**, which focuses on maintaining accurate object trajectories and motion states under ego motion. The present project lies between these traditions: it is not purely a dynamic-object filtering system, but neither is it yet a full object-centric joint optimizer. A useful comparison therefore needs to consider sensor configuration, front-end form, degree of coupling, dynamic-object handling strategy, real-time intention, and engineering complexity.

Table 1: Representative dynamic-scene SLAM and 3D MOT works relative to this project

Work	Sensors / front-end	Coupling	Dynamic handling / real-time stance	Relevance to this project
StaticFusion	RGB-D; dense direct SLAM	Joint camera-motion and segmentation estimation	Weighted background fusion; real-time dense mapping [4]	Shows the value of soft weighting and background preservation rather than only hard rejection.
DS-SLAM	Visual; ORB-SLAM2 + semantic thread	Loosely coupled multi-thread design	Semantic filtering plus motion consistency; real-time oriented [5]	Demonstrates that a modular, multi-thread engineering structure can still improve robustness in dynamic scenes.
DynaSLAM	Visual; ORB-SLAM2 family	Filtering-oriented with static-map recovery	Dynamic detection plus background inpainting; slower but accurate [6]	Clarifies the strengths and limits of treating dynamics mainly as regions to remove.
Conf SLAM-MOT	LiDAR; LiDAR odometry	Tight graph coupling	Confidence-guided association inside optimization [9]	Motivates future confidence-aware object-factor weighting.
LIMOT	LiDAR+IMU; custom LIO	Tight sliding-window factor graph	Joint LiDAR-inertial odometry and MOT in real time [10]	Closest high-level target architecture for upgrading this project beyond topic-level coupling.
IMM-SLAMMOT	Stereo vision; ORB-SLAM2	Tight BA with multiple motion models	Explicit motion-model switching for targets [11]	Highlights that one constant-velocity model is often insufficient in traffic scenes.
Online Dynamic SLAM	Stereo vision; custom VO	Tight incremental smoothing	iSAM2-based online dynamic optimization [12]	Indicates why incremental backends matter for scalability and stability.
DL-SLOT	LiDAR; LiDAR odometry	Tight collaborative graph optimization	Sliding-window collaboration among localization and tracking [13]	Relevant reference for LiDAR-side window management and object factor integration.
AB3DMOT	LiDAR detections; simple KF tracker	No SLAM coupling	3D Kalman filter plus Hungarian matching at high speed [7]	Represents the lightweight tracking philosophy behind many practical MOT pipelines.
CenterPoint	LiDAR; center-based detector/tracker	Detection-tracking integration, not SLAM coupling	Uses center velocity for simple but strong tracking [8]	Important MOT baseline for discussing detector-tracker design choices against MCTrack.

2.1 From Dynamic-Point Rejection to Dynamic-Aware Estimation

StaticFusion, DS-SLAM, and DynaSLAM together define the early evolution of dynamic-scene SLAM [4, 5, 6]. Although they use RGB-D or visual front ends rather than LiDAR-inertial estimation, they establish three ideas that remain relevant here: dynamic regions should be de-

tected explicitly, background structure should be preserved conservatively rather than discarded blindly, and a modular system can still improve robustness without collapsing everything into one monolithic optimizer. Their shared limitation, relative to the present project, is that dynamic objects are treated mainly as regions to suppress or recover around, not as tracked entities whose motion attributes might later help localization. The current system differs by keeping that object-level information alive after filtering.

2.2 Recent SLAMMOT Systems and the Move Toward Joint Optimization

Conf SLAMMOT, LIMOT, IMM-SLAMMOT, Online Dynamic SLAM, and DL-SLOT collectively show the current direction of the field [9, 10, 11, 12, 13]. These works no longer treat SLAM and MOT as isolated black boxes. Instead, they share state, association confidence, motion priors, or optimization windows. The most important common trend is the shift from a pipeline that says “localize first, track later” toward a joint system in which object states, confidence, and ego motion refine one another inside the same estimation procedure.

Conf SLAMMOT is particularly relevant because it emphasizes confidence-guided association inside the optimization process [9]. This matters for the current project because the present backend still uses a comparatively simple object weight based mainly on detection score and track length. The underlying message from Conf SLAMMOT is that confidence should not merely be a threshold for keeping or dropping an object. It should influence the strength of the corresponding factor. That is exactly the direction in which the current backend would need to evolve if it were to become more robust on unstable sequences.

LIMOT is probably the closest reference to the intended long-term target of this project [10]. It integrates LiDAR-inertial odometry and multi-object tracking within a tightly coupled sliding-window framework, making it a direct counterexample to the present topic-level design. Compared with LIMOT, the current project is clearly less unified: FAST-LIO, PointPillars, MCTrack, and the backend still exchange messages rather than sharing one state graph. However, the comparison is not purely negative. Because the current project already publishes the necessary information across module boundaries, including time-stamped poses, LiDAR-frame object observations, world-frame track states, scores, and motion attributes, it has already assembled much of the data that a tighter formulation would require.

IMM-SLAMMOT contributes a different lesson by stressing multiple motion models for dynamic targets [11]. Traffic participants do not all follow one neat constant-velocity pattern. Cars stop, accelerate, and turn; pedestrians behave even less predictably. The present project inherits MCTrack’s practical constant-velocity behavior in online mode, which is sufficient for a first engineering prototype but becomes a liability once those tracked velocities are used as backend constraints. The deterioration observed in some sequences, especially Sequence 0009,

can be interpreted partly through this lens: the backend trusts motion extrapolations that were generated under a simplistic model.

Online Dynamic SLAM is noteworthy because it focuses on incremental smoothing and mapping in dynamic scenes rather than only batch-like sliding windows [12]. Its main relevance here is not the exact front-end modality, which is visual rather than LiDAR-inertial, but the systems lesson: once dynamic information is admitted into the estimation problem, window management, incremental updates, and computational scalability become central design issues. The current ME5400 backend still uses a small ego-only window solved repeatedly with non-linear least squares. That is a reasonable intermediate design, but it does not yet address the long-term scalability question that a real online joint SLAMMOT system must solve.

DL-SLOT is especially instructive on the LiDAR side because it explores collaborative graph optimization between dynamic LiDAR SLAM and 3D object tracking [13]. Relative to the present project, DL-SLOT represents the more mature version of the same general ambition: instead of letting localization and tracking talk only through external interfaces, it shares information at the optimization level. This makes DL-SLOT a useful guide for future backend evolution, particularly in how object-related constraints should be scheduled, weighted, and marginalized over a window rather than merely applied as late corrections.

2.3 Pure 3D MOT Background and the Choice of Tracking Backbone

The MOT literature matters because the usefulness of object information depends on how that information is produced. AB3DMOT and CenterPoint capture two practical design philosophies [7, 8]: one emphasizes lightweight Kalman-style tracking on top of detections, while the other pushes more motion information into the detector-tracker itself. For this project, the key comparison is not which benchmark is strongest, but which interface is most useful downstream. MCTrack is attractive because it exposes track length, fused velocity, velocity change, acceleration, and stable IDs, all of which are directly reused for dynamic weighting and backend gating.

2.4 Comparison and Remaining Gap

Across these works, three broad observations emerge. First, **the field is moving toward tighter coupling**: the strongest recent systems no longer keep SLAM and MOT fully separated. Second, **dynamic-object handling still concentrates on filtering, association, and object-state estimation**; even joint systems do not always exploit dynamic objects as stable geometric support for ego localization. Third, **real-time feasibility and engineering complexity remain decisive**, because synchronization, confidence modeling, and stable optimization become harder as coupling grows tighter.

Relative to that landscape, the current project occupies a deliberately intermediate position. It already implements a genuine bidirectional feedback loop, so it is beyond a one-way cascade, but it remains an engineering-oriented loose-coupling prototype rather than a complete object-centric joint optimizer. Its value lies in having already solved the interface and reproducibility problems that many conceptual designs leave implicit: time-stamped messages, synchronization, coordinate transforms, experiment orchestration, standardized logging, and multi-sequence failure analysis. The remaining gap is precise: the system already has the **information flow** required for a future joint optimizer, but not yet the **state representation** and **factor design** to exploit it fully.

3 Overall System Architecture, PointPillars / MCTrack, and Data Flow

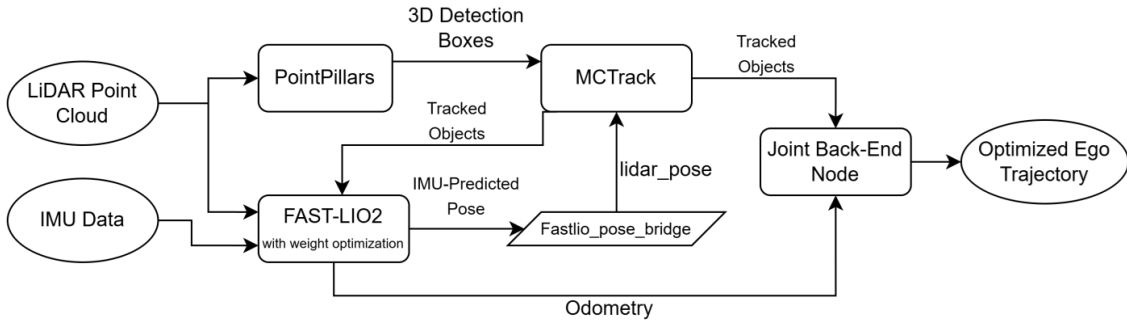


Figure 1: Overall system architecture. It can be interpreted as a three-layer process of perception first, tracking alignment next, and joint correction last.

Figure 1 is easier to read as a data-dependency graph rather than a static block diagram. The first layer performs perception and ego-motion estimation: FAST-LIO publishes poses, while PointPillars publishes 3D detections from the same LiDAR input. The second layer performs temporal and geometric alignment: `fastlio_pose_bridge.py` converts FAST-LIO odometry into the LiDAR-pose interface expected by MCTrack, and MCTrack aligns poses with detections to form coherent object trajectories. The third layer performs feedback and correction: tracked objects are sent back to FAST-LIO for dynamic-point down-weighting and forward to the ego-only backend for windowed trajectory correction.

This organization is necessary because the modules operate at different levels. FAST-LIO works on raw geometry and inertial propagation, PointPillars on learned object hypotheses, MCTrack on temporal association, and the backend on short-horizon ego correction. Keeping these stages explicit avoids blocking behavior and makes it easier to trace both gains and failures to a specific interface.

If the system is understood from the data-flow perspective, the process can be summarized in seven steps:

1. Sensors first acquire raw LiDAR point clouds and IMU data, and feed them simultaneously to FAST-LIO and PointPillars.
2. PointPillars produces the 3D detections of the current frame, indicating which targets are present nearby.
3. FAST-LIO outputs the current ego pose and continuously provides higher-frequency motion predictions.
4. The pose bridge converts these localization outputs into the format directly consumable by the tracking module.
5. MCTrack combines “what was observed” with “how the ego vehicle moved” to complete object association, trajectory updates, and velocity estimation.
6. Tracking outputs are fed back to FAST-LIO so that points likely belonging to dynamic objects can be down-weighted.
7. The same tracking outputs are also sent to the joint backend to further refine the ego trajectory.

This seven-step description already shows why the system must be organized as a directed but delayed loop. Tracks cannot exist before detections are associated, and dynamic-point weighting cannot be computed before a tracked-object message exists. The track state produced at time t therefore influences a later scan rather than the same scan. That one-frame delay is not a bug; it is what keeps the loop online without circular blocking.

3.1 FAST-LIO Front-End

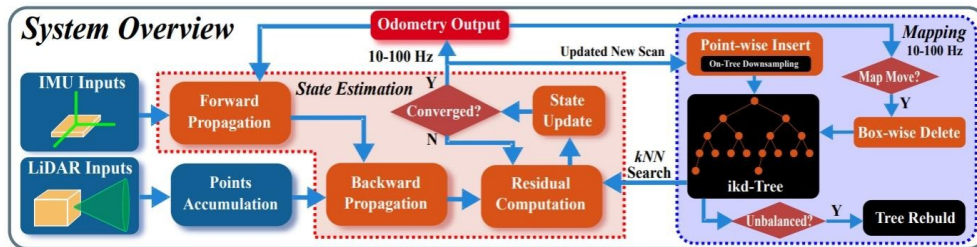


Figure 2: FAST-LIO2 front-end architecture. In this project, object information from MCTrack is injected into the point-cloud preprocessing pipeline to realize adaptive down-weighting of dynamic points.

FAST-LIO is the localization backbone of the entire system and the starting point of the data flow described above. It directly consumes LiDAR point clouds and IMU data from the KITTI rosbag and internally uses a tightly coupled LiDAR-inertial estimation pipeline to output stable ego poses. In this project, FAST-LIO not only generates globally consistent `/Odometry`, but also publishes `/Odometry_imu_predicted` as a high-frequency predicted pose, providing a unified reference for later tracking and backend optimization.

From the perspective of module responsibilities, FAST-LIO plays three roles in the system. First, it provides MCTrack with reliable ego poses, allowing 3D detections to be compensated for ego motion before cross-frame association. Second, it supplies the joint backend with raw odometry constraints so that backend optimization does not drift too far from the front-end estimate. Third, through `preprocess.cpp` and the tracked-object callback path in `laserMapping.cpp`, it receives `/mctrack/tracked_objects`, converts them into internal `DynamicObject` structures, and maps target motion attributes into point-level weights. FAST-LIO therefore acts both as the source of ego-motion information and as the place where object feedback most directly changes scan-level geometry processing.

From an algorithmic viewpoint, the FAST-LIO front-end can be understood as an iterative filter combining IMU propagation with LiDAR geometric correction. Let the system state be $\mathbf{x}_k$ and the IMU input be $\mathbf{u}_k$. The prediction step can be written as

$$\mathbf{x}_{k+1|k} = f(\mathbf{x}_{k|k}, \mathbf{u}_k) + \mathbf{w}_k, \quad \mathbf{P}_{k+1|k} = \mathbf{F}_k \mathbf{P}_{k|k} \mathbf{F}_k^\top + \mathbf{Q}_k, \quad (1)$$

where $\mathbf{P}$ is the state covariance and $\mathbf{F}_k$ is the linearized state-transition matrix. For the j -th point in the current frame, FAST-LIO further constructs a point-to-plane geometric residual using a reference plane in the local map:

$$r_j = \mathbf{n}_j^\top (\mathbf{R}(\mathbf{x}) \mathbf{p}_j + \mathbf{t}(\mathbf{x}) - \mathbf{q}_j), \quad (2)$$

and minimizes through iterative updates

$$\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} \left(\sum_j \|r_j\|_{\square_j^{-1}}^2 + \|\delta \mathbf{x}\|_{\mathbf{P}_{k+1|k}^{-1}}^2 \right). \quad (3)$$

These equations describe the core role of FAST-LIO in this project. The IMU provides high-frequency continuous prediction, while LiDAR point-cloud residuals pull the pose back toward a geometrically consistent solution. This is why FAST-LIO can not only stably output `/Odometry`, but also serve as the spatiotemporal reference for MCTrack.

To feed FAST-LIO outputs into the tracking module, the project implements `fastlio_pose_bridge.py`, which converts `/Odometry` into `/mctrack/lidar_pose` using the configured body-to-LiDAR extrinsic transform and publishes the result as TF. This explicit bridge keeps

localization, tracking, and visualization in the same frame convention. FAST-LIO then freezes the latest tracked objects into a scan-local snapshot before preprocessing each scan, so every point is weighted against a consistent object set rather than a mid-scan message buffer.

3.2 PointPillars Detection Module

PointPillars [2] is wrapped as a ROS node through `MMDet3D/local/ros/nodes/kitti_pointpillars_bag_node.py`, allowing it to directly consume KITTI point clouds from rosbag. Its role is not simply to output raw detections, but to provide the entire system with unified, traceable, and time-stamped 3D detection messages. The project defines a unified message format in `catkin_ws/src/ME5400/msg/Detection3D.msg` and `Detection3DArray.msg`, including frame ID, category, confidence, 2D box, 3D dimensions, LiDAR-frame position, and yaw angle. In this way, PointPillars and MCTrack do not depend on any private third-party message format; they communicate through project-defined message interfaces.

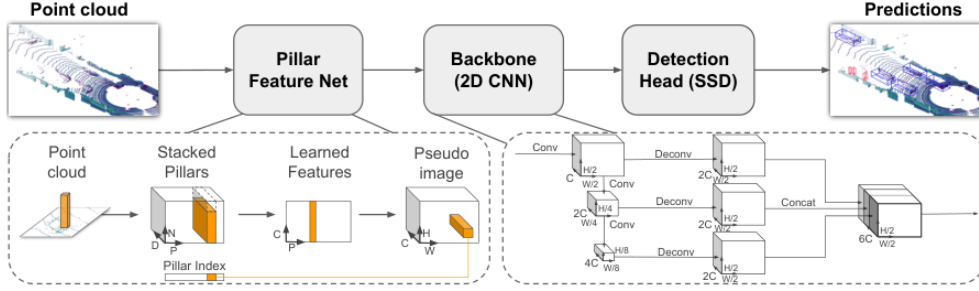


Figure 3: Network architecture from the original PointPillars paper. The network consists of a Pillar Feature Net, a 2D CNN backbone, and an SSD detection head. The image is reproduced from the CVPR 2019 paper by Lang *et al.* [2].

The essence of PointPillars is to convert sparse 3D point clouds into a BEV pseudo-image by encoding vertical pillars and then applying a 2D CNN detection head. This transforms a costly 3D convolution problem into a cheaper 2D one, which is exactly why PointPillars fits the real-time requirements of the current online system.

Written in equations, if the l -th pillar contains the point set $\mathcal{P}_l = \{\mathbf{p}_i\}$, PointPillars first constructs an augmented feature for each point:

$$\tilde{\mathbf{p}}_i = [x_i, y_i, z_i, r_i, x_i - \bar{x}_l, y_i - \bar{y}_l, z_i - \bar{z}_l, x_i - x_l^c, y_i - y_l^c], \quad (4)$$

where $(\bar{x}_l, \bar{y}_l, \bar{z}_l)$ is the mean of points inside the pillar and (x_l^c, y_l^c) is the pillar center. After the Pillar Feature Net, the pillar-level feature can be written as

$$\mathbf{f}_l = \max_{i \in \mathcal{P}_l} \phi(\tilde{\mathbf{p}}_i), \quad (5)$$

which is then scattered into the BEV pseudo-image:

$$\mathbf{I}(u, v) = \begin{cases} \mathbf{f}_l, & (u, v) = \text{index}(l), \\ \mathbf{0}, & \text{otherwise,} \end{cases} \quad (6)$$

Finally, the detection head jointly optimizes classification, box regression, and orientation prediction:

$$\mathcal{L}_{\text{det}} = \beta_{\text{cls}} \mathcal{L}_{\text{cls}} + \beta_{\text{loc}} \mathcal{L}_{\text{loc}} + \beta_{\text{dir}} \mathcal{L}_{\text{dir}}. \quad (7)$$

This also explains why PointPillars fits the project well: efficient 2D convolutions are sufficient to produce time-stamped 3D detections for online ROS publishing.

From an engineering perspective, this design serves three purposes. It preserves calibration consistency, keeps a relatively low detection threshold so that pruning happens later through tracking and gating, and exposes a project-defined message contract that can be reused by MC-Track, visualization, evaluation, and replay. PointPillars deliberately does *not* solve tracking internally; its job is to publish geometrically meaningful 3D hypotheses with timestamps and confidence scores so that later failures can be attributed more cleanly.

3.3 MCTrack Module and Data-Flow Details

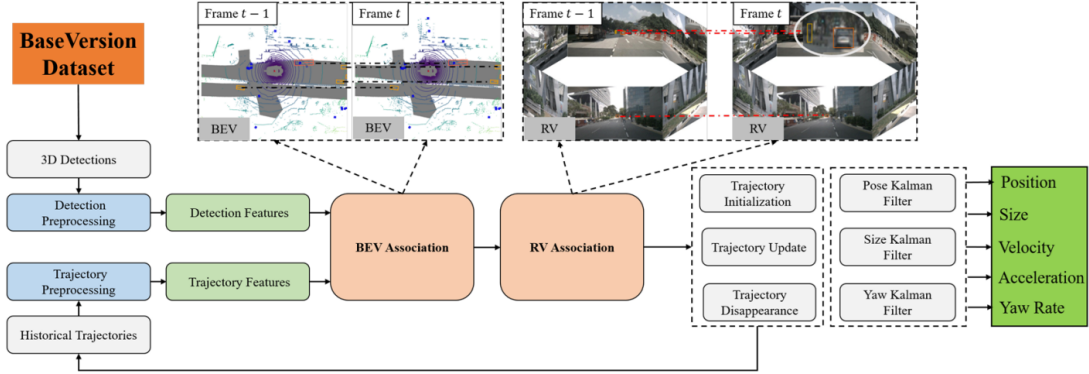


Figure 4: Schematic of the MCTrack 3D MOT framework. Its outputs contain not only position and size, but also velocity, acceleration, trajectory length, and related motion attributes.

The online MCTrack node is implemented in `catkin_ws/src/ME5400/scripts/mctrack_online_node.py`. It subscribes by default to `/mctrack/lidar_pose` and `/detection/lidar_detections`, maintains an internal pose buffer and a queue of pending detections, and processes detections only after a pose with a suitable timestamp is available. This “detections wait for pose, pose drives processing” design avoids the temporal misalignment that arises when messages are processed strictly in order of arrival.

MCTrack publishes both `/mctrack/markers` for visualization and `/mctrack/tracked`

_objects for downstream optimization. In `TrackedObject.msg`, in addition to object pose and size, the message records `track_length`, `velocity`, `velocity_fusion`, `velocity_diff`, `velocity_curve`, and `acceleration`. These fields are crucial to this project because the subsequent dynamic-weight optimization and joint-backend gating depend on motion attributes rather than only on box position.

Under the current `kitti_fastlio.yaml` configuration, the main state estimate of MC-Track in online mode can be approximated as a 2D constant-velocity Kalman filter. If the trajectory state is

$$\mathbf{x}_k = [p_x, p_y, v_x, v_y]^\top, \quad (8)$$

its prediction model is

$$\mathbf{x}_{k|k-1} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}_{k-1|k-1}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{z}_k = \mathbf{H}\mathbf{x}_{k|k-1} + \mathbf{v}_k, \quad (9)$$

followed by the standard Kalman update

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R})^{-1}, \quad \mathbf{x}_{k|k} = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}\mathbf{x}_{k|k-1}). \quad (10)$$

At the data-association level, the current online configuration mainly uses a 3D cost matrix based on predicted boxes together with Hungarian matching:

$$\mathcal{A}^* = \arg \min_{\mathcal{A} \in \Pi} \sum_{(i,j) \in \mathcal{A}} C_{ij}, \quad C_{ij} = 1 - \text{RO-GDIOU}_{3D}(\hat{B}_i, B_j), \quad (11)$$

where $\hat{B}_i$ is the predicted track box and B_j is the current detection box. In other words, FAST-LIO first stabilizes the cross-frame geometry through ego pose, after which MCTrack converts discrete detections into continuous trajectories through motion prediction and global optimal matching. This is the foundation on which later velocity estimation, dynamic weights, and the joint backend become possible.

The buffering and timestamp logic are particularly important. MCTrack stores a finite `pose_buffer` and a queue of pending detections. For each detection message, it looks for poses immediately before and after the detection timestamp; if both exist, it interpolates, otherwise it waits briefly and then falls back to the nearest available pose. This is a practical compromise between strict synchronization and online liveness. MCTrack also preserves both world-frame pose and LiDAR-frame `lidar_pose`, because FAST-LIO needs the LiDAR-frame version for point-box intersection while the backend compares propagated world-frame object motion against the ego state.

Seen this way, PointPillars and MCTrack are not merely a detector and a tracker within the system. They are the modules that generate object-level geometry and motion semantics throughout the full data flow. FAST-LIO provides the stable ego reference, PointPillars provides time-stamped object hypotheses, and MCTrack turns those hypotheses into track states that can be propagated over time. Only after this transformation does it become possible to define dynamic weighting or object-consistency residuals in a meaningful way.

3.4 Temporal Alignment, Coordinate Frames, and Failure Modes

In the optimized online run, rosbag playback publishes only raw LiDAR and IMU topics, while PointPillars, FAST-LIO, MCTrack, and the backend run as separate ROS nodes with different callback timings. Time synchronization is therefore a runtime issue rather than a one-time calibration issue, which is why the pose buffer, pending-detection queue, and one-frame delayed weighting loop are structural parts of the architecture.

The main runtime failure modes are straightforward: **detection latency** forces the tracker to use a nearby rather than truly synchronized pose; **missed detections** shorten tracks and destabilize motion estimates; **track fragmentation** destroys the continuity needed for weighting and backend reuse; and **timestamp or frame inconsistency** can magnify velocity-extrapolation error or even turn an interface bug into an apparent algorithmic failure. The architecture reduces these risks, but does not eliminate them.

4 Engineering Implementation and System Integration

4.1 Overall Code Organization

The repository layout is intentionally organized around integration responsibility rather than around algorithm names alone. The top-level directories that matter most to this report are `catkin_ws/`, `MCTrack/`, `MMDet3D/`, `Scripts/`, and `Results/`. `catkin_ws/` contains the ROS-native part of the system: custom messages, launchable scripts, configuration files, and the modified FAST-LIO package. `MCTrack/` preserves the tracking framework and configuration in a relatively self-contained form so that the online node can import and wrap it rather than reimplementing tracking logic. `MMDet3D/` contains the detector-side code used to wrap PointPillars into the ROS pipeline. `Scripts/` contains orchestration logic, helper utilities, and one-click entry points. `Results/` is not a passive dump folder; it is the standardized experiment archive that the report depends on for every figure and metric.

This partition is beneficial for two reasons. First, it separates heavy third-party or framework-dependent code from project-specific integration code. That makes it easier to upgrade one module without destabilizing all others. Second, it maps closely to the user's

actual workflow. During experimentation, the user rarely edits a detector kernel or Kalman update directly; instead, the user launches scripts, checks logs, inspects ROS topics, records trajectories, and compares result folders. Organizing the repository to match that workflow reduces friction and shortens the feedback cycle during debugging and evaluation.

Table 2: Representative entry scripts of the online system and their functions

Module	Representative entry scripts	Function summary
Online baseline	<code>run_baseline.sh</code> <code>run_fastlio.sh</code>	Executes the pure FAST-LIO online baseline pipeline and records the baseline trajectory in <code>trajectory_baseline.txt</code> .
Online optimization	<code>run_optimized.sh</code> <code>run_pointpillars_node.sh</code> <code>run_mctrack_online_node.sh</code> <code>run_joint_backend_ego.sh</code>	Orchestrates PointPillars, MCTrack, dynamic-weight FAST-LIO, and the joint backend, and outputs the optimized front-end trajectory <code>trajectory.txt</code> together with the backend trajectory <code>trajectory_joint.txt</code> .
One-click online comparison	<code>run_all.sh</code>	Sequentially runs the online baseline and online optimization pipelines and archives the complete runtime logs into the result directory.
Evaluation and archiving	<code>evaluate_trajectories.py</code> <code>odom_to_tum_recorder.py</code>	Uniformly generates quantitative metrics and visualization files, including <code>metrics.json</code> , trajectory-error comparison plots, and ablation summary figures.

Table 3: Main repository directories and their engineering roles

Directory	Representative contents	Engineering role
<code>catkin_ws/</code>	<code>src/ME5400/msg/</code> <code>src/ME5400/scripts/</code> <code>src/fast_lio/</code>	ROS-native workspace for custom interfaces, bridge nodes, MCTrack online wrapper, joint backend, trajectory recording, and the modified FAST-LIO front-end.
<code>MCTrack/</code>	<code>config/kitti_fastlio</code> <code>.yaml</code> , tracker modules	Encapsulates the tracking framework and motion/association configuration used by the online ROS node.
<code>MMDet3D/</code>	PointPillars ROS wrapper and detector utilities	Hosts the detector-side code needed to run PointPillars in the project’s data and topic conventions.
<code>Scripts/</code>	<code>run_all.sh</code> <code>run_baseline.sh</code> <code>run_optimized.sh</code> <code>evaluation/</code>	Provides one-click execution, build/setup helpers, evaluation entry points, and the reproducible experiment workflow.
<code>Results/</code>	<code>metrics.json</code> , trajectories, plots, logs	Serves as the canonical archive for sequence-level outputs used in later analysis, report writing, and regression checking.

4.2 Online System Design

In online mode, `run_all.sh` sequentially executes `run_baseline.sh` and `run_optimized.sh`. This is not just a convenience wrapper. It encodes the fairness policy of the experiments. The

baseline run launches only `roscore`, `FAST-LIO`, and `rosbag` playback so that a pure localization trajectory is recorded as `trajectory_baseline.txt`. The optimized run then launches `PointPillars`, `MCTrack`, `FAST-LIO` in both mode, and the joint backend, while `odom_to_tum_recorder.py` records `/joint_backend/odom` into `trajectory_joint.txt`. Because the scripts operate on the same sequence ID, use the same data source, and archive outputs into the same result folder, they provide a controlled comparison rather than a loose collection of independent runs.

The control flow of the online optimization script is also chosen to minimize race conditions. `PointPillars` starts first because its model initialization is typically the slowest. `MCTrack` starts next and waits until it is ready to receive poses and detections. `FAST-LIO` with dynamic weighting is then launched so that the tracker can immediately begin filling its pose buffer once odometry appears. Finally, the joint backend starts and trajectory recording begins. If this order were reversed, several predictable problems would appear: the recorder might start before any backend topic exists; the tracker might begin queuing detections without usable poses; or the detector might publish too late, causing a burst of stale detections. Figure 5 therefore documents more than a conceptual loop; it records the actual execution logic that keeps the optimized chain stable.

As Figure 5 shows, the online chain is not a synchronous loop in which the current-frame detections immediately modify the current-frame point cloud. Instead, it is an asynchronous loop with a one-frame delay. During `preprocess()`, `FAST-LIO` first freezes the previous `/mctrack/tracked_objects` snapshot and softly down-weights points in the current scan that fall inside dynamic boxes. The current `/Odometry(t)` is then passed through the bridge to `MCTrack`, aligned with `/detection/lidar_detections(t)` from `PointPillars`, and used to update object states. The newly produced `/mctrack/tracked_objects(t)` is then sent to both `FAST-LIO` and the joint backend, but its effect on `FAST-LIO` appears only in the next frame. This design avoids intra-frame blocking, limits lock contention inside the preprocessor, and explains why the current system can form a front-end closed loop while remaining runnable online.

After `rosbag` playback finishes, the script archives `trajectory.txt`, `trajectory_joint.txt`, logs, and evaluation plots into `Results/<seq>_results/Online/`. The benefit of this design is not just cleanliness. It allows a later report or debugging pass to reconstruct exactly what happened for a given sequence: which scripts ran, which trajectories were compared, what the metrics were, and how the front-end and backend plots looked. The limitation, of course, is that the system is still loosely coupled at the topic level: modules share messages rather than a unified graph state. But from an engineering perspective, explicit message passing is precisely what makes the current prototype inspectable and reproducible.

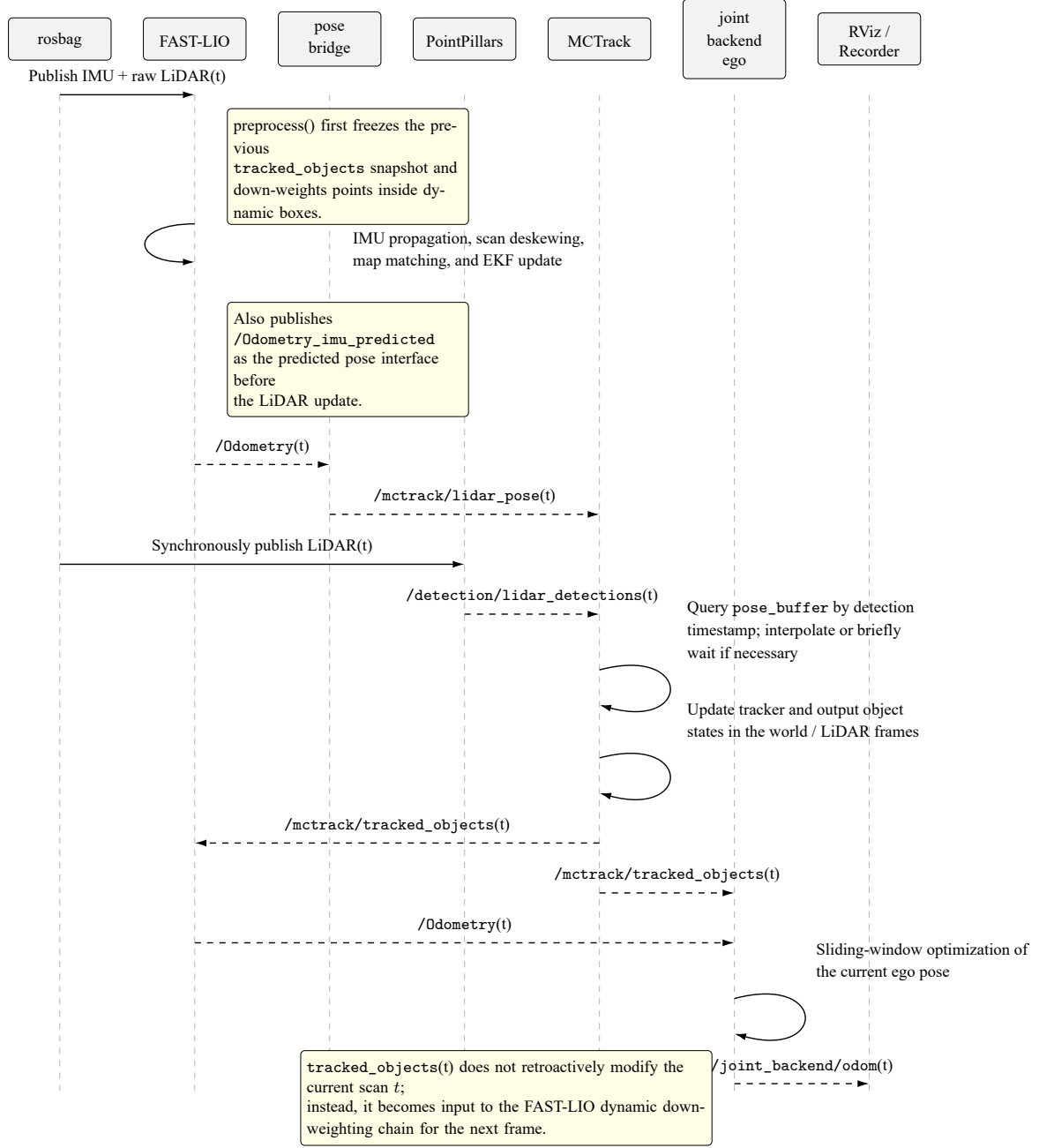


Figure 5: Execution-level temporal logic of the online optimization chain. When FAST-LIO processes the current LiDAR frame, it uses the frozen `tracked_objects` snapshot from the previous frame. The tracking result produced by MCTrack at the current frame mainly enters the dynamic down-weighting chain in the next frame.

4.3 Result Archiving and the Reproducible Experimental Pipeline

The result archive under `Results/<seq>_results/Online/` is best understood as a formal experiment contract. Every online sequence folder contains the baseline trajectory, the optimized front-end trajectory, the joint-backend trajectory when available, a plain-text report, a JSON report, comparison plots, and a full runtime log. This means that quantitative analysis in the report does not depend on manually copied numbers or ephemeral terminal output. It depends on versioned artifacts that can be regenerated by rerunning the scripts. In practice, this is what makes later section-level expansion possible: ATE, RPE, Tr, Rot, failure sequences, and plots all originate from the same archived source.

The evaluation path reinforces this reproducibility goal. `evaluate_trajectories.py` loads the recorded TUM trajectories, associates them with KITTI ground truth, computes ATE and RPE, aligns trajectories for visualization, and computes KITTI-style relative translation and rotation metrics when enough matched path length exists. Because the same script is used for every baseline versus optimized comparison, the report avoids a common course-project weakness in which each sequence is evaluated slightly differently. Even when different sequences expose different available span lengths, as in short versus long trajectories, the evaluation rule is still consistent.

This reproducibility is not a secondary convenience; it is part of the intellectual contribution of the project. A course project that claims a localization gain but cannot rerun its own sequences, recover its own metrics, or explain its own failures has limited value. By contrast, the present system can be launched with fixed scripts, can record trajectories in a standard format, can regenerate figures, and can tie every conclusion in the report back to a concrete archived artifact. That is why engineering reproducibility deserves to be treated as a first-class outcome rather than an appendix-level detail.

4.4 ROS Topics, Message Interfaces, and Integration Difficulties

The module boundaries in this repository are enforced primarily through ROS topics and project-defined messages. The raw inputs are `/kitti/velo/pointcloud` and `/kitti/oxts/imu`. FAST-LIO publishes `/Odometry` and `/Odometry_imu_predicted`. The bridge node republishes `/mctrack/lidar_pose`. PointPillars publishes `/detection/lidar_detections`. MCTrack publishes `/mctrack/tracked_objects` and `/mctrack/markers`. The backend publishes `/joint_backend/odom`. Each topic corresponds to a clear semantic responsibility, and the custom message definitions ensure that detector outputs, tracked-object states, and backend inputs remain interpretable throughout the pipeline.

The custom messages are especially useful for avoiding hidden coupling. `Detection3D.msg` stores frame index, class, confidence, 2D box, 3D dimensions, LiDAR-frame location,

and yaw. `TrackedObject.msg` extends that into an object state useful for feedback, including track length, world-frame pose, LiDAR-frame pose, fused velocity, velocity difference, curved velocity, and acceleration. This explicitness is what makes the dynamic-weight module and backend feasible. The weighting code can read LiDAR-frame pose, track length, score, and acceleration directly. The backend can compare local observations to world-frame predictions without trying to reconstruct state from an opaque tracker object. In short, the message layer is where the project translates algorithm-specific internals into stable cross-module contracts.

The integration was still difficult in several ways. Dependency isolation is one challenge: MMDet3D, ROS, MCTrack, and FAST-LIO each bring their own runtime assumptions, environment variables, and build expectations. Startup order is another challenge, because the whole optimized pipeline is sensitive to whether detector, tracker, and backend are ready before rosbag playback begins. Logging is a third challenge, addressed by redirecting full script output to per-sequence logs. Result naming and archival are a fourth challenge: if trajectory files were not renamed and moved consistently, later evaluation would silently compare the wrong outputs. These are mundane issues individually, but together they determine whether an integrated robotics system behaves like a controlled experiment or like an unstable demo.

5 Dynamic Weight Optimization

5.1 Design Motivation and Code Location

The central design question of this module is simple: when a tracked object overlaps a region of the point cloud, should the system keep those points, remove them, or reduce their influence continuously? The current project chooses the third option. Instead of turning dynamic-point handling into a binary segmentation problem, it treats tracked objects as evidence that the reliability of nearby geometry should be reduced by a degree that depends on motion state and confidence. This is motivated by both geometry and engineering. Geometrically, hard removal can erase too much useful structure in traffic scenes where large vehicles occupy a significant share of the scan and where roadside geometry may already be sparse. Engineering-wise, hard removal is brittle under false positives, unstable boxes, and partial occlusion; a slightly wrong box can delete a large amount of valid structure. Soft weighting is more conservative and therefore more appropriate for a first integrated prototype.

The current implementation lives mainly in `catkin_ws/src/fast_lio/src/preprocess.cpp`, with tracked-object conversion in `catkin_ws/src/fast_lio/src/laserMapping.cpp`. After FAST-LIO receives `/mctrack/tracked_objects`, each object is converted into an internal `DynamicObject` that stores its LiDAR-frame center, bounding-box half-sizes, local yaw representation, timestamp, and a scalar weight. Importantly, the conversion uses the object's LiDAR-frame pose rather than its world-frame pose, because point-in-box tests must be carried

out in the current LiDAR coordinate system. This makes the weighting stage tightly tied to actual preprocessing rather than to some abstract global map representation.

Compared with earlier versions, the default parameters for dynamic weighting in the current code are explicitly fixed as follows:

- reference velocity $v_{\text{ref}} = 10.0$;
- reference acceleration $a_{\text{ref}} = 4.0$;
- velocity penalty coefficient $\alpha_v = 0.5$;
- acceleration penalty coefficient $\alpha_a = 0.3$;
- detection-score penalty coefficient $\alpha_s = 0.2$;
- minimum weight $w_{\text{min}} = 0.2$;
- bounding-box margin $xy = 0.5 \text{ m}$, $z = 0.5 \text{ m}$;
- object timeout 1.0 s, with a minimum trajectory-length threshold of 3.

Each parameter reflects an engineering compromise. The reference velocity and acceleration define what should count as clearly dynamic, the minimum weight prevents the scheme from collapsing into hard deletion, the box margin compensates for imperfect boundaries, and the timeout plus minimum track length prevent stale or immature trajectories from influencing the scan.

5.2 Weight Computation

Suppose that a tracked target has velocity v , acceleration a , and detection score s . The code first normalizes these quantities:

$$\hat{v} = \text{clamp}\left(\frac{v}{v_{\text{ref}}}\right), \quad \hat{a} = \text{clamp}\left(\frac{a}{a_{\text{ref}}}\right), \quad \hat{s} = \text{clamp}(s). \quad (12)$$

It then constructs the penalty term

$$p = \alpha_v \hat{v} + \alpha_a \hat{a} + \alpha_s \hat{s}, \quad (13)$$

and computes the dynamic weight as

$$w = \text{clip}(1 - p, w_{\text{min}}, 1). \quad (14)$$

The resulting interpretation is intuitive. Faster targets with larger acceleration and higher detection score receive smaller point weights. If a trajectory is short, nearly static, or weakly scored, the system preserves a larger weight. The trajectory-length guard is particularly important: if `track_length` is below threshold, `weightFromAttributes()` returns 1.0, so the object cannot influence the point cloud yet.

The choice of velocity, acceleration, and score reflects the information the tracker can already provide reliably. This is not a fully probabilistic uncertainty model, but it is a practical heuristic built from interpretable motion attributes already present in `TrackedObject.msg`.

5.3 From Tracked Objects to Point-Level Weights

The transformation from object tracks to point weights happens in one scan-local pipeline. The tracked-object callback converts each object into an internal representation using `lidar_pose.position`, margin-expanded half sizes, a compact yaw representation, and a scalar weight computed from fused speed, acceleration, detection score, and track length. When a new scan enters `process()`, FAST-LIO freezes the latest tracked objects into an active snapshot. Each point then calls `computePointWeight()`, checks whether it lies inside any non-expired dynamic box, and takes the minimum weight among the boxes that cover it. This means `tracked_objects(t)` does not retroactively modify the scan at time t ; it becomes the consistent object set used when preprocessing a later scan.

The weight becomes effective directly in point-cloud preprocessing. For each input point, if `computePointWeight()` finds dynamic-box overlap, the point’s intensity is updated to the smaller of the original intensity and the dynamic weight:

$$I'_j = \min(I_j, w_j), \quad (15)$$

where I_j is the original point intensity and w_j is the computed dynamic weight. This reuses the existing FAST-LIO point representation to carry reliability information downstream without rewriting the registration machinery.

Compared with outright deleting dynamic points, this strategy is more robust to box-boundary error, detection jitter, and partially dynamic regions; it preserves some geometric support when static background is limited; and it differentiates between a clearly moving mature track and a barely confirmed object without a hard semantic switch. This graded behavior is one reason the front-end loop is more stable than the current backend.

5.4 Boundary Cases and Parameter Sensitivity

Several boundary cases clarify why soft weighting is preferable in the current system. **False positives** and **occlusion-driven box instability** can overlap static structure or jitter in extent; under hard deletion those errors would erase geometry completely, whereas soft weighting only reduces influence and only after track maturity and motion checks pass. **Temporally uncertain cases**, including temporarily stopped vehicles and stale trajectories, are handled more conservatively through the combination of score, acceleration, minimum track length, and timeout.

Although the project does not include a formal parameter sweep, the archived sequences support a qualitative sensitivity discussion. If v_{ref} is too low or w_{min} too small, the module behaves too much like hard filtering; if v_{ref} is too high or w_{min} too large, dynamic weighting becomes too weak to matter. The current values appear to be a practical middle ground: strong enough to help on drift-prone sequences such as 0010 and 0020, but conservative enough to avoid systematic degradation across all seven online runs.

The broader effect is improved scan matching and local-map consistency without demanding that object-derived information already be accurate enough to support strong backend corrections. This balance is why dynamic weighting is the most stable gain source in the current system.

6 Joint Backend Optimization

6.1 The Ego-Only Sliding-Window Problem

The joint backend is implemented in `catkin_ws/src/ME5400/scripts/joint_backend_ego_node.py`, with configuration stored in `catkin_ws/src/ME5400/config/joint_backend_ego.yaml`. Its purpose is intentionally transitional. Rather than solving a full object-centric SLAM-MOT problem immediately, it tests a narrower hypothesis at manageable engineering cost: can stable tracked objects provide enough additional structure to refine the ego trajectory, even if object states are not jointly optimized? The answer matters because it determines whether the project should invest later effort in a richer backend at all. An ego-only backend is therefore not the final goal; it is a diagnostic stepping stone.

The current configuration makes this compromise explicit. The default window length is 8 frames. The object-factor coefficient is $\lambda_{\text{obj}} = 0.12$. The score threshold is 0.6, the track-length threshold is 5, the maximum allowed timestamp difference is 0.1 s, and the maximum allowed velocity jump is 2.0 m/s. The minimum object weight is 0.03. The backend uses a Huber robust loss with scale 0.5, optimizes at 10 Hz, and falls back to $[0.20, 0.20, 0.10]$ if the incoming FAST-LIO covariance is invalid. None of these choices is arbitrary. Together they define a backend that is intentionally conservative about when to trust object information and

intentionally lightweight about how much state to optimize.

The optimization variable for each frame in the window is written as

$$\mathbf{p}_i = [x_i, y_i, \psi_i]^\top, \quad (16)$$

meaning that only planar position and yaw are retained per frame. This choice is restrictive, but it is reasonable for two engineering reasons. First, the tracked-object information used by the backend is already mostly 2D in its motion semantics: fused velocities and association stability are primarily meaningful in the ground plane. Second, retaining only the most critical ego variables keeps the optimization small enough that it can run repeatedly online without becoming the dominant compute bottleneck. The obvious trade-off is that object errors cannot be jointly estimated and can only be used as external observations; if a track is wrong, the backend has no internal variable representing that object’s state to absorb the error.

Choosing this ego-only formulation first was therefore a deliberate risk-control strategy. A full joint graph would have required object poses, velocities, data-association uncertainty, marginalization policy, and probably more careful motion-model design. By contrast, the current backend can be implemented, tested, and stress-analyzed using information already produced by MCTrack. This is why it is best described as a prototype of the backend direction rather than as the backend solution itself.

6.2 Residual Modeling and the Gating Strategy

The joint backend includes two major residual types. The first is odometry-consistency residuals: relative poses between adjacent FAST-LIO frames are treated as the primary constraints of the sliding window, ensuring that optimization does not deviate excessively from the original odometry. The second is object-observation consistency residuals: for each frame, the backend matches the nearest tracking snapshot in time and retains only objects that satisfy score, track-length, velocity-jump, and time-synchronization gates. For each retained object, the code first extrapolates its world-coordinate position to the current frame timestamp using its velocity and then projects it into the ego local frame used during optimization, yielding the predicted observation z_{ik}^{pred} . The measured observation z_{ik}^{meas} comes from the LiDAR-local pose stored in `TrackedObject`. The object residual is therefore

$$r_{ik}^{\text{obj}} = z_{ik}^{\text{pred}} - z_{ik}^{\text{meas}}. \quad (17)$$

The overall optimization objective is

$$\min_{\{\mathbf{p}_i\}} \sum_{i=2}^N \|r_i^{\text{odom}}\|_{W_i}^2 + \sum_{i=1}^N \sum_{k \in \mathcal{V}_i} w_{ik} \rho\left(\|r_{ik}^{\text{obj}}\|_2^2\right), \quad (18)$$

where $\rho(\cdot)$ is a Huber or Cauchy robust kernel and $\mathcal{V}_i$ denotes the set of objects that pass the gates. In the current code, the object-factor weight is approximately written as

$$w_{ik} = \max\left(w_{\min}^{\text{obj}}, \sqrt{\lambda_{\text{obj}}} \cdot (0.5 \hat{s}_{ik} + 0.5 \hat{\ell}_{ik})\right), \quad (19)$$

that is, it is jointly determined by normalized score and normalized trajectory length.

This residual structure is intentionally asymmetric. The odometry residuals are treated as the stable backbone of the window. The object residuals act as optional corrections whose role is to nudge the solution when object evidence is mature and consistent. The backend therefore does not ask tracked objects to dominate the estimate. It asks them to provide a secondary source of agreement about how the ego platform should be situated relative to the observed scene. That asymmetry is sensible for a prototype, because it limits how much damage a bad track can do. At least in theory, the odometry chain should still anchor the window even when some object factors are imperfect.

In practice, the gating strategy determines whether this theory holds. The backend first discards any track whose score is below threshold, whose track length is too short, whose velocity jump exceeds the allowed maximum, or whose timestamp differs too much from the current odometry state. These conditions correspond closely to the actual failure channels of tracked-object reuse. Low score suggests uncertain detection quality. Short track length suggests immature temporal evidence. Large velocity jumps often indicate a track update that is numerically unstable or semantically wrong. Excessive timestamp mismatch makes velocity extrapolation unreliable even if the track itself is good. The gating policy is therefore not arbitrary conservatism. It is the encoded belief about which tracked objects are mature enough to be treated as weak moving anchors for the ego state.

6.3 Window Update Logic and Practical Behavior

The backend is driven by a timer callback rather than by optimization on every message. It stores recent odometry states and track snapshots in ring buffers, builds the latest window when enough frames exist, and avoids re-optimizing the same window end stamp. The optimized result is applied as a correction to the latest raw odometry pose, preserving FAST-LIO roll and pitch while adjusting planar position and yaw. The backend therefore behaves more like an online corrector than a full smoothing engine.

This explains the mixed experimental behavior observed later. When object information is clean and baseline drift is significant, the backend can provide useful low-frequency correction. When object information is noisy, a few bad residuals can remain visible because the window is small and object states are not jointly estimated.

6.4 Value and Limitations of the Joint Backend

The engineering value of this backend is that it validates a key proposition: even when object states are not treated as explicit optimization variables, stable tracked objects can still act as temporary moving anchors that impose additional constraints on the ego trajectory. In other words, the system has already moved beyond the view that dynamic targets are merely noise to be filtered out. The backend therefore serves as a proof-of-direction. It shows that tracked objects can enter the estimation problem as positive information, not only as sources of point down-weighting.

Its limitations, however, are equally clear. Because object states come from point estimates of the tracker and are extrapolated through velocity propagation, any noise in object velocity, detection boxes, or association results can be injected back into the ego pose. The backend also lacks richer factor diversity: there are no explicit object-state variables, no stronger confidence model than score and track length, and no sophisticated data-association uncertainty representation. Moreover, the motion model implicit in the tracker is still relatively simple, so extrapolated object positions are not always trustworthy enough to constrain ego motion.

Sequence 0009 is the clearest failure case. In online mode, the baseline ATE RMSE is 1.6831 m and the front-end loop slightly improves it to 1.6675 m, but the joint backend degrades the ATE RMSE to 3.3082 m, while RPE RMSE rises from 0.1787 to 1.9514. The most plausible interpretation is not that object information is useless in principle, but that several weak links aligned negatively in this sequence. Velocity extrapolation likely accumulated error over short object horizons. Timestamp mismatch, even if within the nominal gate, likely reduced the physical meaning of propagated object states. Gating based on score, track length, and velocity jump was not sufficient to suppress all harmful factors. Once those residuals entered a small ego-only window, the optimizer had no object-state variables or stronger uncertainty model to absorb the inconsistency. This makes Sequence 0009 far more convincing as a diagnostic example than a generic statement that the backend is “sometimes unstable.”

The correct conclusion is therefore measured. The backend direction is valid, because some sequences do show useful drift correction and because the formulation already demonstrates how object information can enter ego optimization. But the present ego-only implementation remains an engineering prototype rather than a robust joint backend. A reasonable evolution path is clear: add better confidence modeling, incorporate object-state variables directly, enrich the factor set beyond simple object observations, and eventually move toward an object-centric backend in which ego motion and object motion are estimated together instead of one being used only as a correction source for the other.

7 Experimental Analysis

7.1 Experimental Setup and Result Sources

The experiments in this report are uniformly based on the archived online results already stored under Results/. The analyzed online sequences are 0003, 0006, 0007, 0009, 0010, 0013, and 0020. All numerical values are taken from the corresponding `metrics.json` files, and the discussed plots correspond to `evaluation_result.png`, `evaluation_joint_backend.png`, and `ablation_bar.png`. This section therefore does not present new experiments performed during report writing; it consolidates and interprets the experiment artifacts already generated by the reproducible pipeline.

In online mode, three configurations are compared: Baseline, FAST-LIO+MCTrack, and Joint Backend. The baseline is pure FAST-LIO. FAST-LIO+MCTrack adds pose-assisted tracking and tracking-informed dynamic weighting but evaluates the front-end optimized trajectory directly. Joint Backend further evaluates the ego-only backend output recorded from `/joint_backend/odom`. This ordering is analytically useful because it isolates the effect of the front-end loop from the additional effect, positive or negative, of the backend stage.

The main evaluation metrics serve different purposes and should not be conflated. **ATE RMSE** measures the root-mean-square global trajectory deviation after alignment to ground truth, so it is most informative about accumulated drift and overall path consistency. **RPE RMSE** measures short-horizon relative motion disagreement, so it is more sensitive to local frame-to-frame stability. **KITTI relative translation error** Tr summarizes percentage translation error over path segments of fixed length, while **KITTI relative rotation error** Rot summarizes angular drift per 100 m. A method can therefore improve ATE without improving RPE, or improve translation consistency without improving rotation consistency. This distinction is important for interpreting the current backend, which sometimes behaves like a coarse drift corrector but not a uniformly better local optimizer.

The seven selected sequences cover a useful range of operating conditions. Sequence 0010 and Sequence 0020 are representative of high-dynamic or drift-prone settings in which front-end feedback has room to show a large gain. Sequences 0003 and 0006 are shorter or more moderate cases in which the front-end improves but the backend can over-correct. Sequences 0007 and 0013 represent relatively stable cases where improvements are limited but generally consistent. Sequence 0009 is the critical failure case in which the backend degrades severely. Organizing the analysis by these scene characteristics is more informative than discussing each sequence as an isolated number.

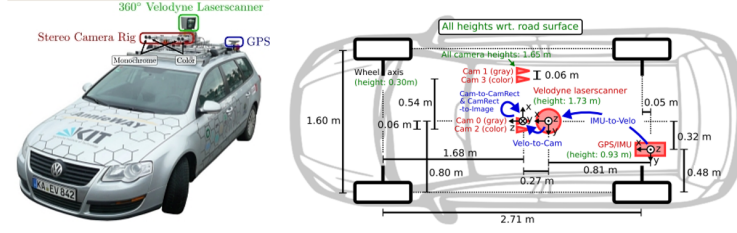


Figure 6: KITTI data-collection platform. All experiments in this report are based on archived KITTI Tracking results.

7.2 Online Experiments: Aggregate Results

Table 4 summarizes ATE and RPE for all online sequences. The most important aggregate observation is immediate: FAST-LIO+MCTrack improves ATE RMSE over the baseline on all seven online sequences. This 7/7 behavior is stronger evidence than any single large success case because it shows that the front-end loop is robust across varied conditions. By contrast, Joint Backend further reduces ATE on only four sequences, and two of those gains are effectively neutral in practice. The backend therefore does not exhibit the same cross-sequence stability as the front-end loop.

Table 4: Summary of online ATE/RPE results. Improvement percentages are computed relative to Baseline; positive values indicate lower error.

Sequence	Baseline ATE	F+M ATE	F+M Improvement	Joint ATE	Joint Improvement	RPE RMSE (B/F/J)
0003	0.5294	0.4513	+14.75%	0.5875	-10.98%	0.2049 / 0.1809 / 0.2393
0006	0.5238	0.4710	+10.09%	0.5576	-6.45%	0.1324 / 0.0869 / 0.1677
0007	2.3128	2.2393	+3.18%	2.2393	+3.18%	0.1716 / 0.1683 / 0.1683
0009	1.6831	1.6675	+0.93%	3.3082	-96.56%	0.1787 / 0.1651 / 1.9514
0010	3.7651	0.8680	+76.95%	0.9632	+74.42%	0.2774 / 0.2281 / 0.2334
0013	0.6047	0.5816	+3.82%	0.5816	+3.82%	0.1495 / 0.1392 / 0.1392
0020	11.7323	6.7938	+42.09%	6.8829	+41.33%	0.2250 / 0.2245 / 0.2270

The ATE table also shows why the backend should not be judged only by average improvement. Sequence 0010 is a major success for both front-end and backend, but Sequence 0009 is a catastrophic backend failure. The systems lesson is therefore that the front-end gain is stable while the backend gain is conditional. The RPE columns reinforce this distinction: the front-end often improves or preserves local stability, whereas the backend can introduce large local inconsistencies when object-derived corrections are unreliable.

Table 5 reports KITTI relative translation and rotation errors in online mode. Here again the front-end loop behaves more consistently than the backend. FAST-LIO+MCTrack improves relative translation error on six of the seven evaluable sequences and relative rotation error on four, while Joint Backend improves relative translation on only three and often worsens relative rotation. This is exactly the pattern expected from a backend that sometimes corrects

long-horizon drift but does not yet guarantee smoother local geometry.

Table 5: Summary of online KITTI relative errors. Rot is given in deg/100m, with the order Baseline / F+M / Joint.

Sequence	Baseline Tr(%)	F+M Tr(%)	F+M Improvement	Joint Tr(%)	Joint Improvement	Rot (B/F/J)
0003	1.4794	1.3516	+8.63%	1.6856	-13.94%	1.1509 / 1.1569 / 1.3699
0006	4.1614	4.0605	+2.42%	4.3990	-5.71%	12.8416 / 11.6291 / 16.0675
0007	3.9042	3.9267	-0.57%	3.9267	-0.57%	4.9702 / 5.0234 / 5.0234
0009	2.5630	2.5281	+1.36%	4.2200	-64.65%	3.2425 / 3.2294 / 3.7550
0010	1.3884	0.9773	+29.61%	1.1140	+19.76%	1.1274 / 1.1368 / 1.5713
0013	2.4374	2.3741	+2.60%	2.3741	+2.60%	2.3026 / 2.2419 / 2.2419
0020	3.2245	2.3967	+25.67%	2.4302	+24.63%	1.1894 / 1.1748 / 1.2326

Among the online results, the two strongest front-end success cases are Sequences 0010 and 0020. For Sequence 0010, ATE RMSE decreases dramatically from 3.7651 m to 0.8680 m, an improvement of 76.95%. For Sequence 0020, ATE RMSE drops from 11.7323 m to 6.7938 m, an improvement of 42.09%. These are not minor refinements. They show that when baseline drift is substantial and dynamic objects strongly affect the scan, the combination of reliable pose support for tracking and soft dynamic-point down-weighting can substantially improve localization. Figure 7 further illustrates this behavior for Sequence 0020, where the optimized trajectory is visibly closer to ground truth than the baseline.

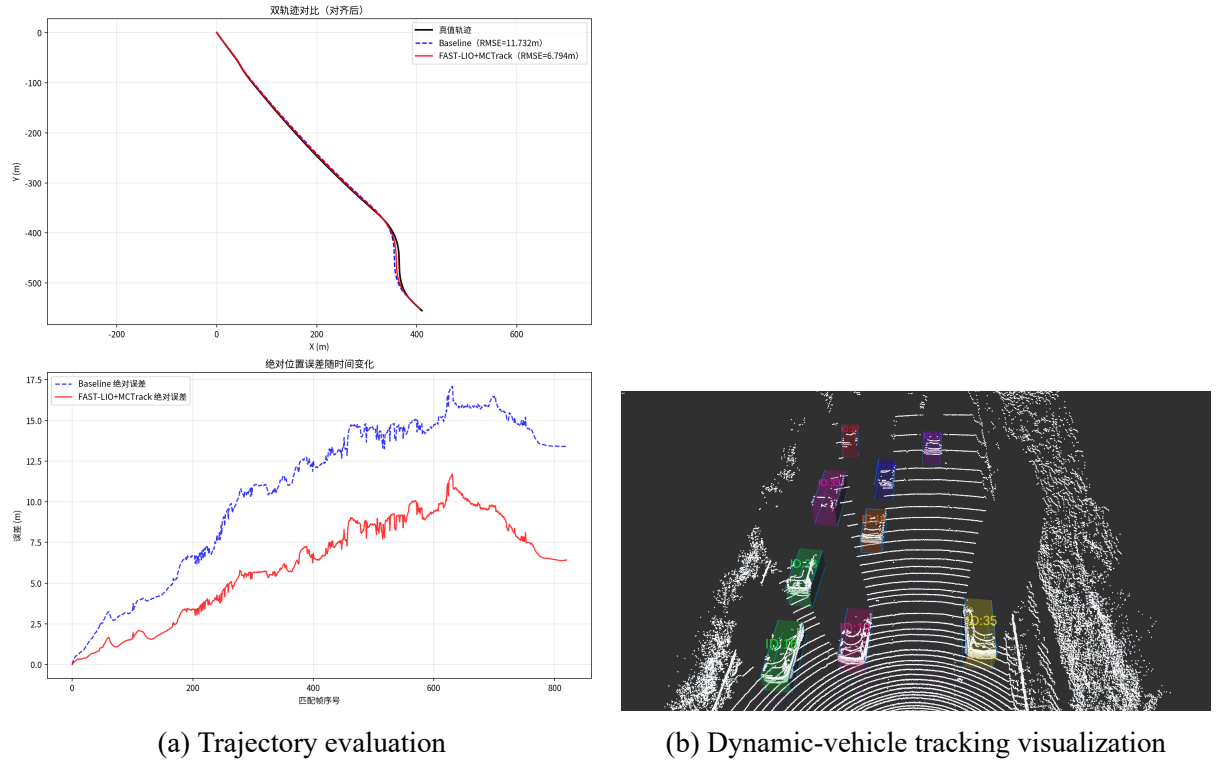


Figure 7: Representative online results for Sequence 0020. The left panel shows that the red FAST-LIO+MCTrack trajectory is clearly closer to the ground truth than the blue baseline. The right panel visualizes dynamic-vehicle tracking on the same sequence.

7.3 Integrated Analysis and Failure Modes

High dynamic scenes with large baseline drift: Sequences 0010 and 0020. These two sequences are the clearest demonstrations of the current system’s front-end value. In Sequence 0010, the baseline already suffers significant global deviation, and the front-end loop reduces ATE RMSE from 3.7651 m to 0.8680 m while also lowering RPE RMSE from 0.2774 to 0.2281. The joint backend remains good relative to baseline but is slightly worse than the front-end output, which suggests that most of the gain has already been captured before backend correction. Sequence 0020 shows a similar but longer-horizon pattern. The front-end loop cuts ATE RMSE from 11.7323 m to 6.7938 m and improves KITTI translation error from 3.2245% to 2.3967%. The backend preserves much of this global gain, but RPE changes only marginally. The most plausible explanation is that these sequences suffer from dynamic-scene contamination and global drift strongly enough that better front-end weighting yields a large benefit, while the backend has little clean additional information left to exploit.

Moderate dynamic scenes: Sequences 0003 and 0006. These two sequences show a subtler but extremely important pattern. The front-end loop improves both ATE and RPE: Sequence 0003 improves ATE RMSE by 14.75%, and Sequence 0006 improves it by 10.09%, with especially clear RPE improvement in 0006 from 0.1324 to 0.0869. Yet in both sequences the backend degrades performance, pushing ATE above baseline and noticeably worsening RPE and KITTI relative errors. This indicates that the front-end improvement is not merely a by-product of easier sequences. The front-end helps even in moderate cases. The backend failure in these sequences likely reflects over-correction: the baseline and front-end are already reasonably good, so a small number of noisy object factors can do more harm than good. In other words, the backend currently lacks a sufficiently strong activation criterion for deciding when object-derived correction is worth the risk.

Low-dynamic or static-structure-rich scenes: Sequences 0007 and 0013. These sequences are valuable because they test whether the proposed loop remains safe when the problem is not strongly dynamic. The answer is mostly yes for the front-end. Sequence 0007 shows a small ATE gain from 2.3128 m to 2.2393 m, and Sequence 0013 shows a modest but consistent ATE and RPE gain. The backend is nearly identical to the front-end in both cases, indicating that object-derived correction neither adds much nor causes serious damage. This is a useful outcome. It suggests that the front-end weighting loop does not require highly dramatic dynamic content in order to remain stable, while also showing that the backend has little positive leverage when the baseline is already comparatively well constrained by static structure.

Backend failure case: Sequence 0009. Sequence 0009 deserves to stand alone because its failure is qualitatively different from the moderate degradations seen elsewhere. The front-end loop makes a small but real improvement, reducing ATE RMSE from 1.6831 m to 1.6675 m and RPE RMSE from 0.1787 to 0.1651. The backend then collapses: ATE RMSE rises to 3.3082 m,

ATE max error reaches 40.3063 m, KITTI translation error worsens from 2.5630% to 4.2200%, and RPE RMSE jumps to 1.9514. This pattern points to a localized but severe inconsistency being injected into the ego estimate. The likely root causes are exactly those discussed in the backend chapter: velocity extrapolation error, insufficiently strict gating, and timing mismatch between odometry and tracked objects. Sequence 0009 therefore functions as the anchor case proving that the backend is not merely “less good than the front-end” but can actively destabilize the trajectory if object evidence is trusted too much under the current design.

7.4 From Observation to Explanation

Taken together, the scene-grouped results support a clear causal interpretation. When the baseline is mainly hurt by dynamic-object contamination in the front-end, the current system helps reliably because dynamic weighting attacks that mechanism before registration and mapping errors accumulate. This is why the front-end loop improves all seven sequences and delivers the largest gains in drift-prone dynamic cases. When the baseline is already relatively stable, the backend has less room to help and more chance to inject noise through imperfect tracked-object constraints.

The difference between ATE and RPE trends is also instructive. On some sequences, the backend preserves a better global path shape while offering little or no RPE improvement, which is consistent with a low-frequency correction mechanism rather than a uniformly better local optimizer. The front-end remains stable because its use of object information is conservative and local, whereas the backend converts tracked-object information into stronger ego constraints and therefore demands much better synchronization, confidence modeling, and motion prediction quality.

7.5 Experimental Fairness and Reporting Discipline

Finally, the experimental conclusions are strengthened by the consistent comparison protocol across all reported sequences. The same archived data source, trajectory format, evaluation script, and result-directory convention are used for baseline, front-end, and backend comparisons. This matters when interpreting small gains versus large failures, because it reduces the chance that the trends come from post-processing differences rather than actual system behavior.

The main experimental conclusion of the report is therefore unambiguous. The current project has already validated a useful and reproducible front-end closed loop between localization and tracking. That loop is the most stable source of gain in the entire system. The backend direction is promising but still immature, and its failures are informative rather than accidental. For a course project, this is a strong result: the system does not merely show that some im-

provement exists, but identifies where the improvement is robust, where it is fragile, and why the difference occurs.

8 Conclusion and Future Plan

This report expands the ME5400 project into a fuller technical narrative while keeping the conclusions aligned with the actual repository state. The project already achieves something meaningful: it builds a complete bidirectional loop among FAST-LIO, PointPillars, and MCTrack; it exposes that loop through explicit ROS topics and project-defined messages; it provides one-click reproducible experiments; and it evaluates the resulting trajectories quantitatively across seven KITTI Tracking sequences. In that sense, the work is already more than a simple component survey or a one-off demonstration. It is a functioning engineering validation platform for studying how localization and tracking can help one another in dynamic driving scenes.

The most important verified conclusion is also the simplest. The most stable and reliable gain in the current system comes from the front-end loop of “FAST-LIO pose aiding MCTrack” and “MCTrack feeding dynamic weights back into FAST-LIO.” This front-end configuration improves ATE RMSE on all seven reported online sequences and improves KITTI translation error on most of them. The dynamic-weight module is therefore not a secondary embellishment; it is currently the most mature technical contribution in the system. It improves scan matching and local-map consistency in a conservative, robust way that is compatible with online operation.

The report is equally clear about what has *not* yet been solved. The current joint backend is not a robust object-centric optimizer. It is an ego-only sliding-window prototype that tests a valid idea under simplified assumptions. It can help on drift-dominated sequences, but it can also degrade badly when object velocity extrapolation, time alignment, or gating are imperfect. Sequence 0009 is the key reminder that reusing tracked objects as positive localization constraints is substantially harder than merely using them for soft front-end suppression. This is not a contradiction of the project hypothesis. It is the exact evidence needed to refine that hypothesis into a stronger next-stage design.

The future work can therefore be prioritized rather clearly. **First**, the backend should evolve from an ego-only corrector into an **object-centric joint backend** in which object states are optimized together with ego pose instead of entering only as external observations. This is the most important upgrade because it directly addresses the current inability to absorb object-state error inside the optimization. **Second**, the system needs **stronger confidence modeling and association strategy**. Detection score and track length are useful first heuristics, but they are not sufficient for deciding when an object factor should influence ego motion strongly. Confidence-aware weighting, more explicit motion consistency checks, and richer

data-association diagnostics are all needed. **Third**, the optimization should move toward **incremental or more efficient windowed updates**. If the object-centric formulation becomes richer, the current lightweight least-squares loop will not remain enough. **Fourth**, the project should be evaluated on **more datasets and more complete ablations**, not mainly to increase paper-style volume, but to test whether the same front-end versus backend pattern persists beyond the current KITTI subset.

From the perspective of course value, this is a strong and honest outcome. The project has already integrated multiple complex modules into one coherent workflow, proved that a localization-tracking feedback loop can produce stable front-end gains, and exposed the real engineering gaps that block a stronger joint backend. That combination of working system, reproducible experiments, and transparent failure analysis is precisely what makes the current prototype useful for continued research. If the next iteration succeeds in promoting today’s message-level feedback into a true object-centric estimation graph, the project can evolve naturally from a course-scale engineering validation into a more substantive research contribution on dynamic-scene LiDAR-inertial SLAMMOT.

References

- [1] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “FAST-LIO2: Fast direct LiDAR-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [2] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. CVPR*, 2019, pp. 12697–12705.
- [3] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? The KITTI vision benchmark suite,” in *Proc. CVPR*, 2012, pp. 3354–3361.
- [4] R. Scona, M. Jaimez, Y. R. Petillot, M. Fallon, and D. Cremers, “StaticFusion: Background reconstruction for dense RGB-D SLAM in dynamic environments,” in *Proc. ICRA*, 2018.
- [5] C. Yu, Z. Liu, X. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, “DS-SLAM: A semantic visual SLAM towards dynamic environments,” in *Proc. IROS*, 2018.
- [6] B. Bescos, J. M. Facil, J. Civera, and J. Neira, “DynaSLAM: Tracking, mapping and inpainting in dynamic scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, 2018.
- [7] X. Weng, J. Wang, D. Held, and K. Kitani, “3D multi-object tracking: A baseline and new evaluation metrics,” in *Proc. IROS*, 2020, pp. 10359–10366.
- [8] T. Yin, X. Zhou, and P. Krahenbuhl, “Center-based 3D object detection and tracking,” in *Proc. CVPR*, 2021, pp. 11784–11793.

- [9] S. Fang and H. Li, “LiDAR SLAMMOT based on Confidence-guided Data Association,” *arXiv preprint arXiv:2412.01041*, 2024.
- [10] Z. Zhu, J. Zhao, K. Huang, X. Tian, J. Lin, and C. Ye, “LIMOT: A Tightly-Coupled System for LiDAR-Inertial Odometry and Multi-Object Tracking,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6600–6607, 2024.
- [11] P. Tian and H. Li, “Visual SLAMMOT Considering Multiple Motion Models,” *arXiv preprint arXiv:2411.19134*, 2024.
- [12] J. Morris, Y. Wang, and V. Ila, “Online Dynamic SLAM with Incremental Smoothing and Mapping,” *arXiv preprint arXiv:2509.08197*, 2025.
- [13] X. Tian, Z. Zhu, J. Zhao, G. Tian, and C. Ye, “DL-SLOT: Tightly-Coupled Dynamic LiDAR SLAM and 3D Object Tracking Based on Collaborative Graph Optimization,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1017–1027, 2024.

Acknowledgment

As I complete this report for the ME5400 project, I would like to express my sincere gratitude to everyone who has supported me throughout this work. This report is not only a summary of the project itself, but also a meaningful record of my learning, research, and engineering practice during my study.

First of all, I would like to thank all the teachers in the College of Design and Engineering at the National University of Singapore for their teaching, guidance, and academic support during my master's study. Their patience and dedication have greatly deepened my understanding of both research and engineering practice.

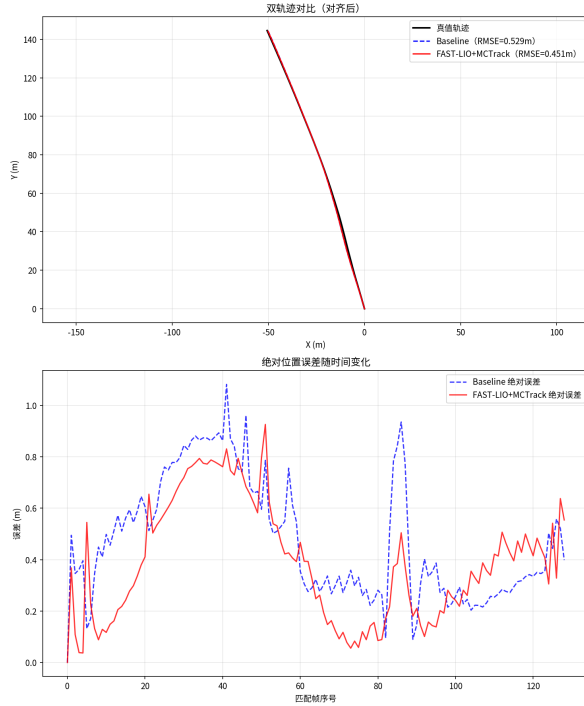
I am especially grateful to my supervisor, Professor Marcelo H. Ang Jr., for his valuable guidance and support throughout this project. I would also like to sincerely thank Professor Teo Chee Leong for his time and thoughtful evaluation of this work. My heartfelt thanks also go to Han Yuhang, whose guidance was of great help during this research, and to every member of our research group for their companionship, discussions, and encouragement.

Finally, I would like to express my deepest respect and gratitude to my family and friends. Their love, trust, and support have always given me strength, especially during difficult and uncertain moments.

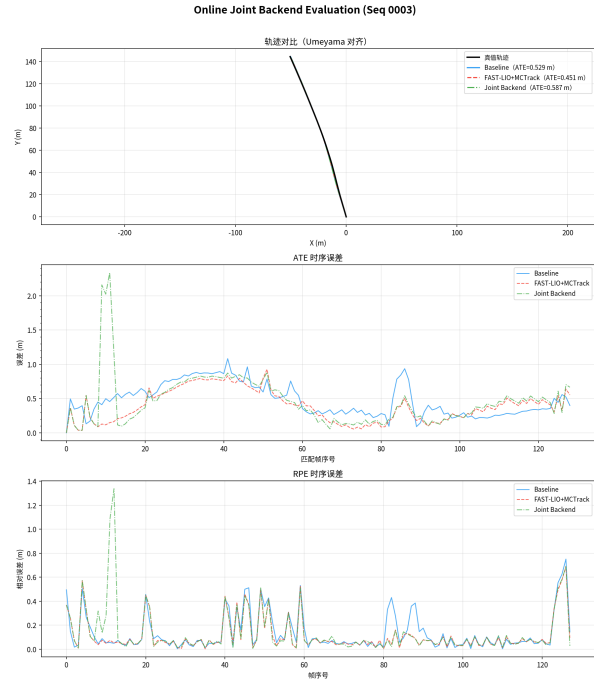
As I move forward into future work and life beyond school, I will carry this kindness with me and continue my journey with sincerity, gratitude, and hope.

Appendix: All Online Experimental Figures

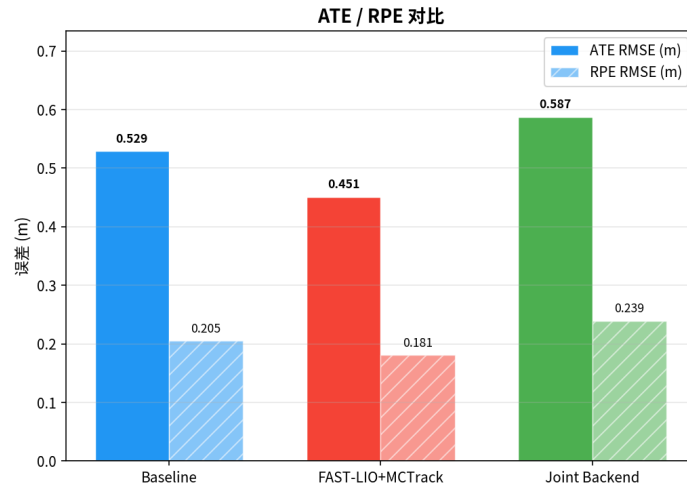
This appendix inserts all online result figures by sequence. Each sequence includes the online front-end comparison, the joint-backend evaluation, and the bar-chart comparison.



(a) Online front-end comparison

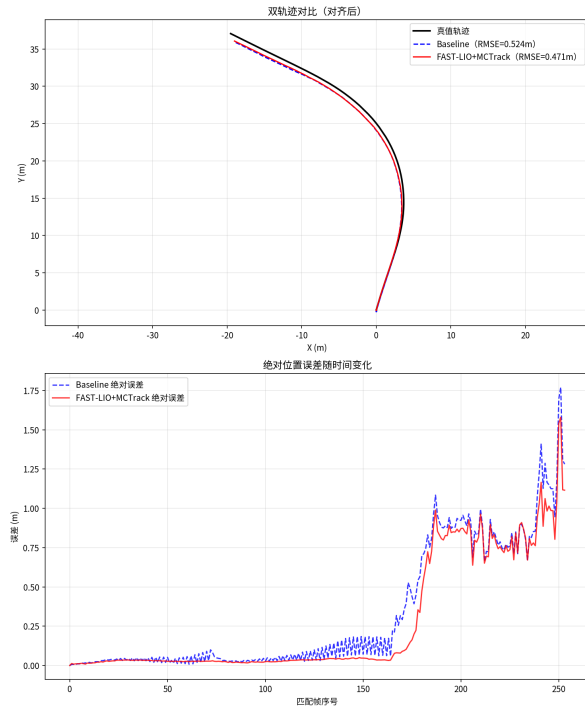


(b) Joint backend evaluation

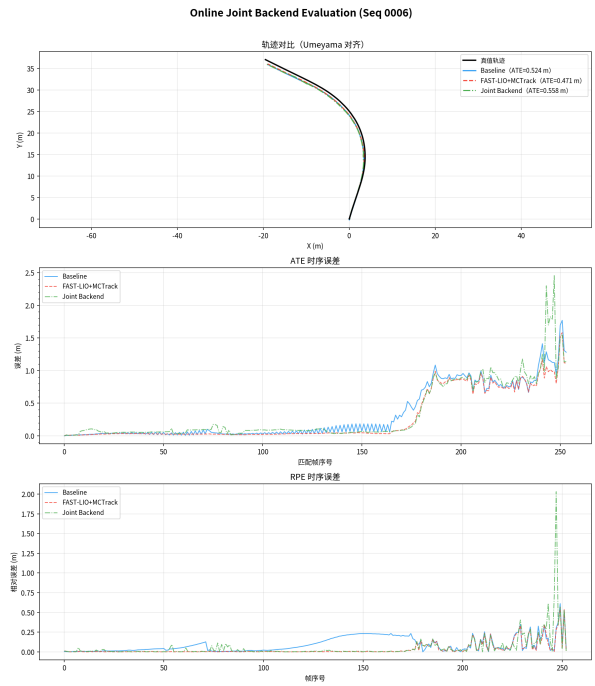


(c) ATE/RPE bar comparison

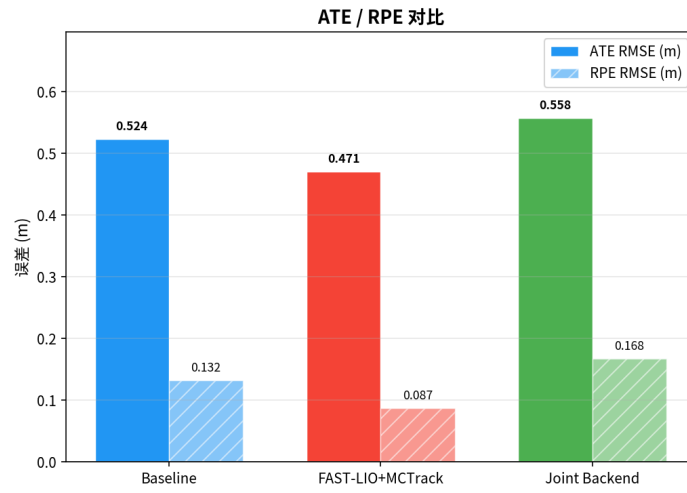
Figure 8: Complete online experimental results for Sequence 0003.



(a) Online front-end comparison

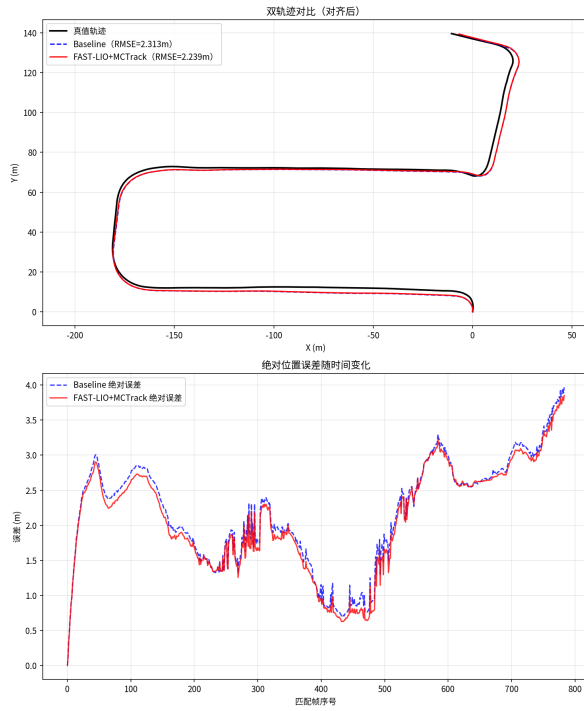


(b) Joint backend evaluation

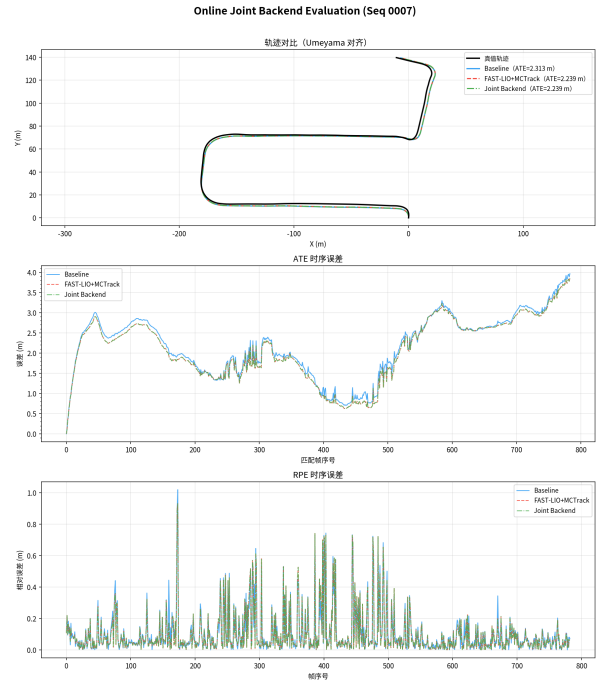


(c) ATE/RPE bar comparison

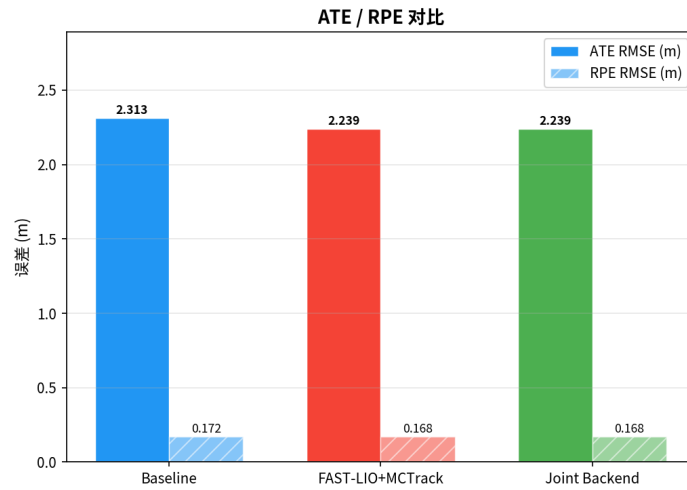
Figure 9: Complete online experimental results for Sequence 0006.



(a) Online front-end comparison

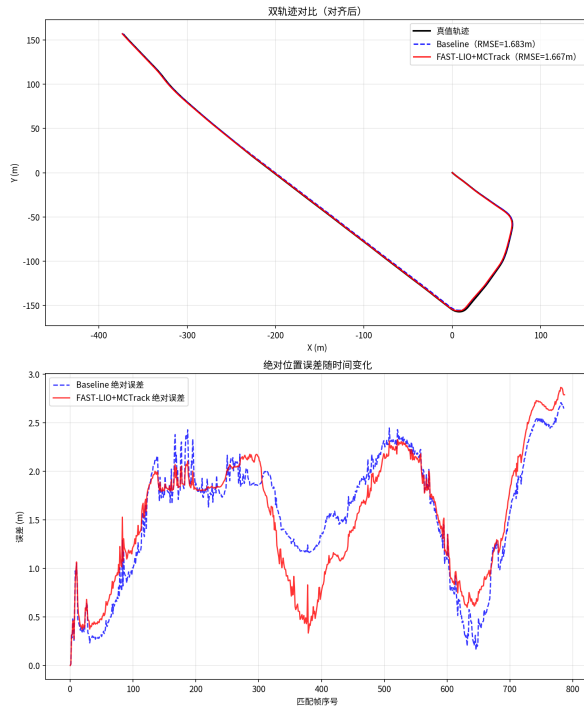


(b) Joint backend evaluation

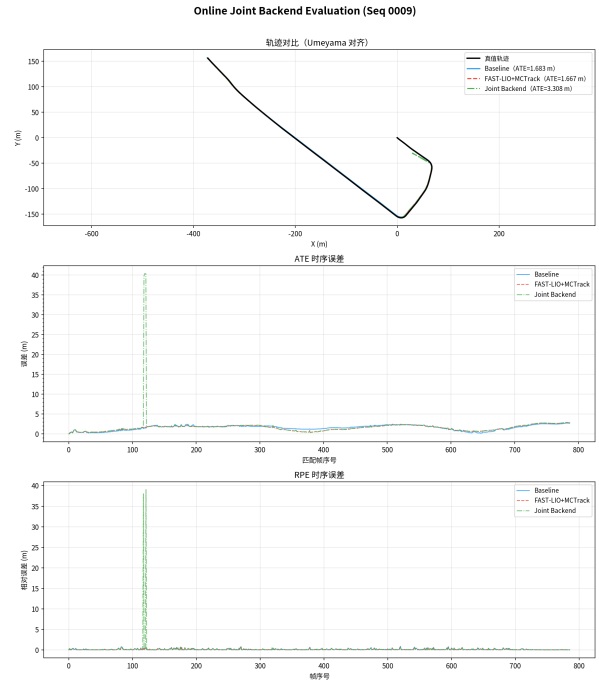


(c) ATE/RPE bar comparison

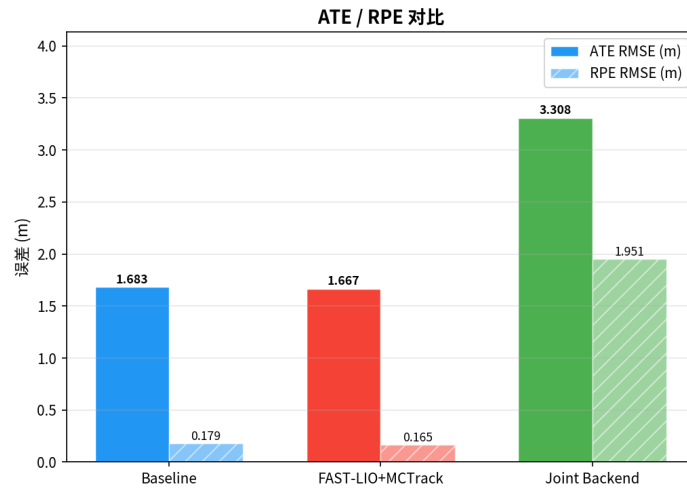
Figure 10: Complete online experimental results for Sequence 0007.



(a) Online front-end comparison

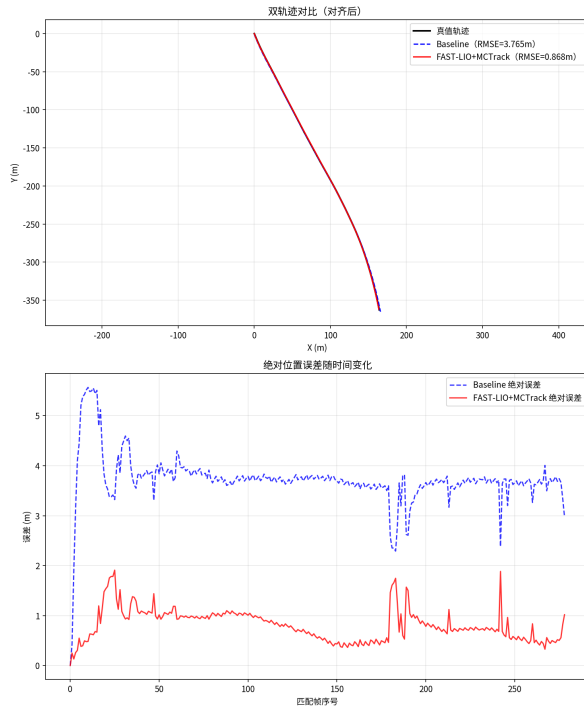


(b) Joint backend evaluation

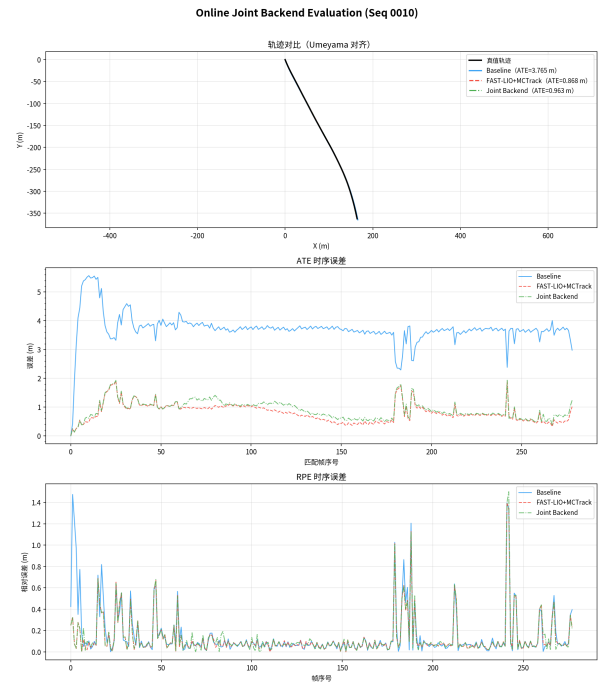


(c) ATE/RPE bar comparison

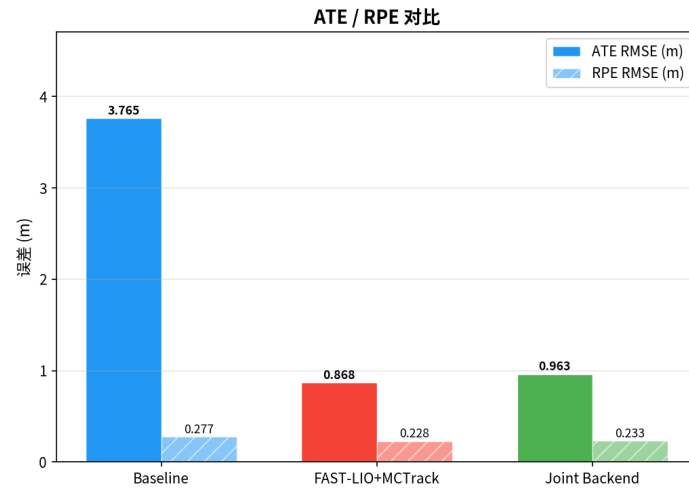
Figure 11: Complete online experimental results for Sequence 0009.



(a) Online front-end comparison

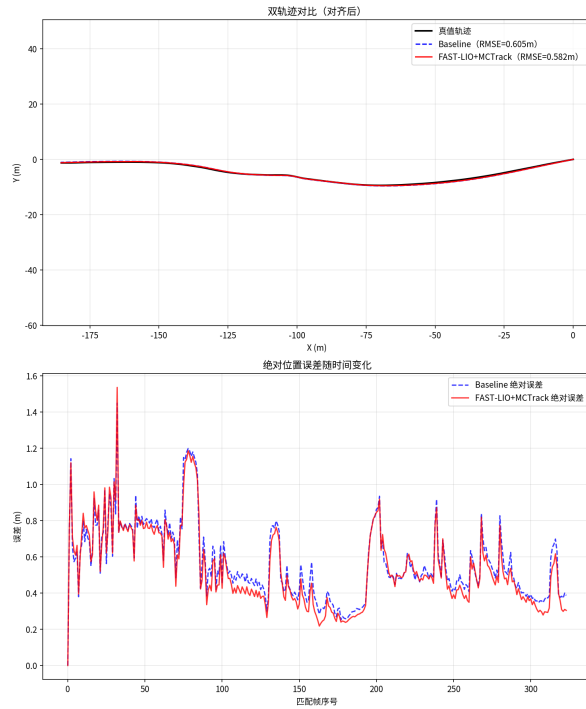


(b) Joint backend evaluation

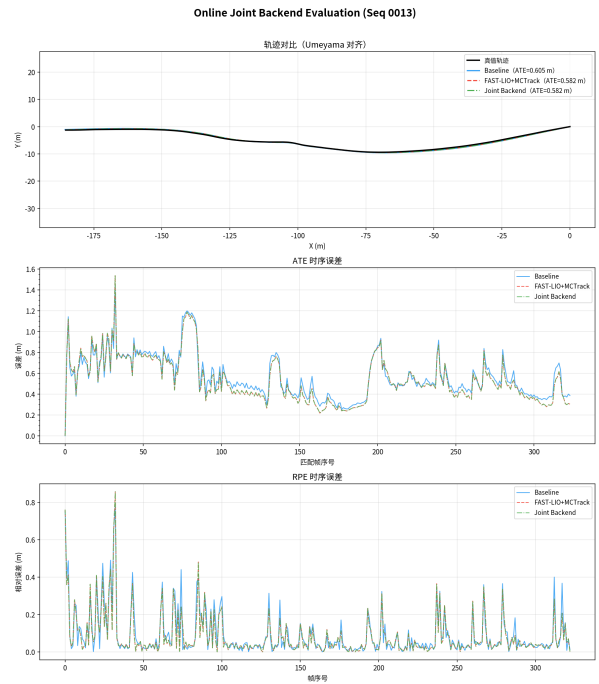


(c) ATE/RPE bar comparison

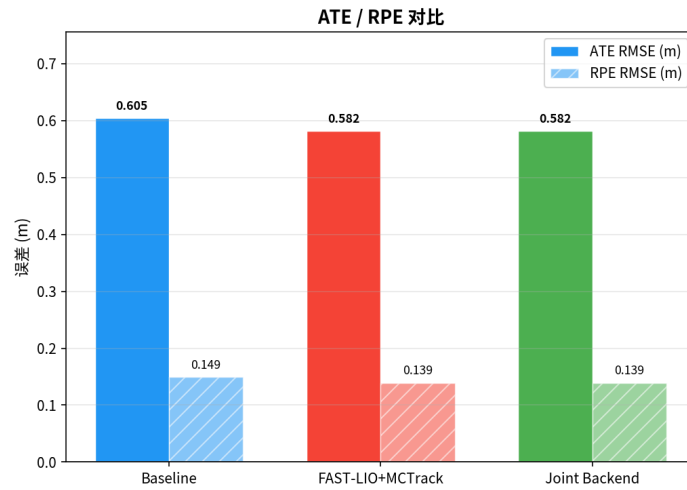
Figure 12: Complete online experimental results for Sequence 0010.



(a) Online front-end comparison

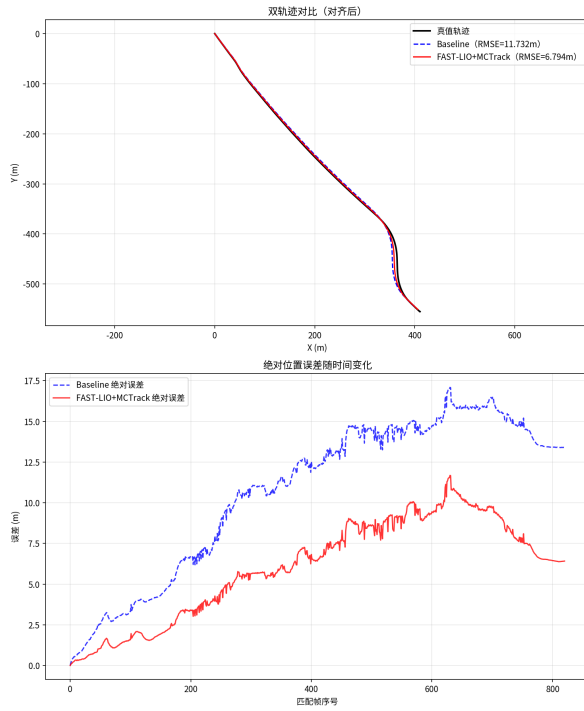


(b) Joint backend evaluation

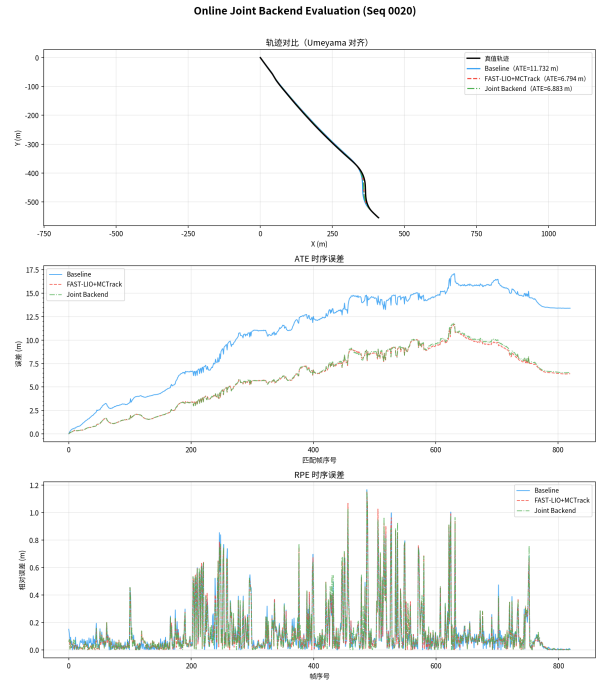


(c) ATE/RPE bar comparison

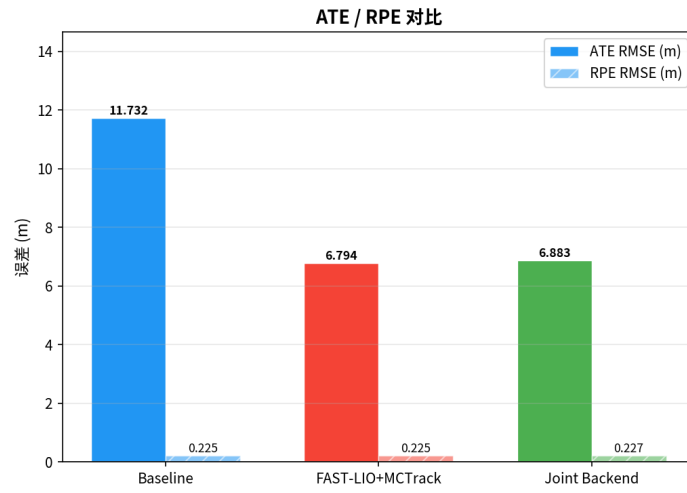
Figure 13: Complete online experimental results for Sequence 0013.



(a) Online front-end comparison



(b) Joint backend evaluation



(c) ATE/RPE bar comparison

Figure 14: Complete online experimental results for Sequence 0020.